

EduNexus OS — Release 0.1

Institutional Work Management Foundation

The objective of this release is to establish the **first usable product slice** around the core value chain:

Communication → Request → Workflow → Approval → Task → Evidence → Completion

Release scope

Area	Release 0.1
Identity & access	✓
Organization hierarchy	✓
People/directory	✓
Dashboard	✓
Announcements & circulars	✓
Correspondence	✓
Unified inbox	✓
Requests/cases	✓
Dynamic forms	✓
Workflow & approvals	✓
Tasks & actions	✓
SLA & escalation	✓
Notifications	✓
Documents & attachments	✓
Audit trail	✓
Basic reports	✓
Arabic/English	✓
Mobile/PWA	✓

Area	Release 0.1
Meetings/committees	 Release 0.2
Decisions/policies	 Release 0.2
Accreditation/strategy	 Release 0.3
Advanced AI	 Release 0.3

1. The first production product

The first release should revolve around a new concept:

My Work

Every employee should open EduNexus and immediately see:

- What do I need to know?
- What do I need to approve?
- What do I need to do?
- What am I waiting for?
- What is overdue?

The home experience therefore becomes:

EduNexus OS										Search			Profile		
Home			Good morning, Dr. Ahmed												
My Work			Faculty of Engineering												
Inbox															
Requests															
Correspond.			12		4		7		2						
Tasks			Tasks			Approvals			Requests			Overdue			

Communicate					
Documents	Priority Work				
Reports					
	● Approve faculty workload	Today			
	● Reply to correspondence	Today			
	● Review purchase request	Tomorrow			
	Recent Communications				
	New academic circular				
	Faculty council notification				

This is much more valuable than building another generic university portal.

2. Core navigation

Release 0.1 gets a deliberately focused navigation:

HOME

MY WORK

└─ My Tasks

└─ My Approvals

└─ My Requests

└─ Waiting For

└─ Overdue

COMMUNICATION

- └─ Announcements
- └─ Circulars
- └─ Correspondence
- └─ Inbox
- └─ Sent

REQUESTS

- └─ New Request
- └─ My Requests
- └─ Assigned Requests
- └─ All Requests

DOCUMENTS

- └─ My Documents
- └─ Shared Documents
- └─ Templates
- └─ Archive

REPORTS

- └─ My Performance
- └─ Department
- └─ Workflow
- └─ Executive

ADMINISTRATION

- └─ Organization

- └─ Users
 - └─ Roles
 - └─ Forms
 - └─ Workflows
 - └─ Notifications
 - └─ Templates
 - └─ Audit
-

3. Release 0.1 core modules

M01 — Identity & Organization

The institution administrator creates:

Tenant

- └─ University
 - └─ Campus
 - | └─ Faculty
 - | | └─ Department
 - | | └─ Program
 - | | └─ Unit
 - | └─ Center
 - └─ Institute

Each person receives:

- User account
- Employee/student classification
- Organizational unit
- Position
- Manager
- Roles

- Permissions
 - Delegation rules
 - Notification preferences
-

4. M02 — Communication Center

Three communication levels:

Announcement

General institutional information.

Circular

Formal institutional communication.

Directive

Communication that creates an explicit action.

Example:

Circular: Examination procedures for Fall 2026.

versus:

Directive: Department heads must submit examination schedules by 20 September.

The second one automatically creates actionable work.

5. M03 — Correspondence

This becomes one of the product's strongest features.

Incoming correspondence

Received



Registration



Classification



Assign



Action



Follow-up



Response



Close

Correspondence record

Correspondence No.

Type

Subject

Source

Recipient

CC

Date

Priority

Classification

Confidentiality

Due Date

Assigned To

Department

Status

Attachments

Related Request

Related Task

Related Document

Audit History

6. M04 — Unified Inbox

Instead of forcing users to inspect five different modules:

Inbox

└─ Correspondence

└─ Approvals

└─ Tasks

└─ Requests

└─ Notifications

└─ Mentions

The user can filter:

All | Action Required | Approval | Correspondence | Task | Request | Overdue

And every item tells the user:

What is required from me?

7. M05 — Request Center

This is the transactional heart of Release 0.1.

Users click:

+ New Request

and select:

Academic

Administrative

HR

Finance

Procurement

IT

Facilities

Student Affairs

Research

Quality

Other

The form changes dynamically according to the request type.

8. Dynamic Form Builder

Administrators can create:

Form

- |— Sections
- |— Fields
- |— Validation Rules
- |— Conditional Fields
- |— Attachments
- |— Permissions
- |— Workflow
- └— SLA

Supported fields:

- Text
- Long text
- Number
- Currency
- Date
- Date/time
- Dropdown

- Multi-select
- Radio
- Checkbox
- User
- Department
- Organization
- Document
- Attachment
- Signature
- Table/grid

Example:

Purchase Request

Request Information

- └ Requester
- └ Department
- └ Purpose
- └ Required Date

Items

- └ Item
- └ Quantity
- └ Estimated Cost
- └ Justification

Attachments

Approval

└─ Department Head

└─ Finance

└─ Authorized Executive

9. M06 — Workflow Engine

The workflow designer is visual:

START

|



FORM

|



VALIDATE

|



Department Head Approval

|

└─ Reject ──▶ CLOSED

|



Finance Approval

|

└─ Reject ──▶ CLOSED

|



Dean Approval

|



TASK

|



COMPLETION

|



VERIFY

|



CLOSE

Supported workflow nodes:

- Start
- Form
- Assignment
- Approval
- Parallel approval
- Conditional branch
- Notification
- Task
- Timer
- SLA
- Escalation
- Webhook
- API call
- Document generation
- End

A manager receives:

Approval Required

Faculty workload plan

Submitted by: Department Head

Department: Computer Science

Total workload: 312 hours

Required action: Approve

[Approve]

[Reject]

[Request Changes]

[Delegate]

The approval must capture:

- decision
- timestamp
- user
- organization
- comments
- delegated authority, if applicable
- related documents
- audit event

No more:

"Please approve this by email."

11. M08 — Tasks & Actions

Every approved workflow can create tasks.

Task

- └─ Task ID
- └─ Title
- └─ Description
- └─ Owner
- └─ Department
- └─ Priority
- └─ Start Date
- └─ Due Date
- └─ SLA
- └─ Status
- └─ Progress
- └─ Dependencies
- └─ Evidence
- └─ Comments
- └─ Audit

Statuses:

New

Assigned

In Progress

Waiting

Blocked

Completed

Verified

Cancelled

12. M09 — SLA & Escalation

Example:

Task assigned

|



48 hours

|

| — completed → close

|



Reminder

|



72 hours

|



Manager escalation

|



Executive escalation

The escalation rules are configurable per workflow.

13. M10 — Notification Engine

The architecture:

Business Event



Notification Rule



Audience Resolver



Priority



Channel



Template



Delivery Queue



Email / Push / SMS / Teams



Delivery Receipt

Priority:

Priority	Channels
----------	----------

FYI	In-app
-----	--------

Normal	In-app + Email
--------	----------------

Important	In-app + Email + Push
-----------	-----------------------

Urgent	In-app + Email + Push + SMS
--------	-----------------------------

Emergency	All authorized channels
-----------	-------------------------

14. M11 — Documents

Release 0.1 includes institutional document management, but deliberately avoids trying to become a full enterprise ECM.

Core functions:

- Upload
- Download
- Preview
- Version
- Share
- Classify
- Search
- Attach to transaction
- Approve
- Archive
- Retention
- Access control

Most importantly:

documents become associated with business transactions.

For example:

Purchase Request

|
├─ Quotation.pdf
├─ TechnicalSpecification.docx
├─ Approval.pdf
└─ PurchaseDecision.pdf

15. M12 — Audit

The audit model is:

Who + What + When + Where + Why + Before + After + Source

Example:

Event:

Approval Approved

User:

Dean / 1042

Transaction:

REQ-2026-001245

Previous:

Pending

New:

Approved

Timestamp:

2026-09-11 14:32

IP:

...

Reason:

Approved according to delegated authority.

We should **not** literally log every UI click as business audit data. Product analytics and technical telemetry should remain separate from legally/business-relevant audit events.

16. First killer workflow

The first showcase workflow should be:

Faculty Administrative Request

Employee



Create Request



Department Review



└─ Changes Required



└─ Employee



Department Approval



Faculty Administration



Dean Approval



Task Generation



Execution





Evidence Upload



Verification



Closed

This single workflow demonstrates almost the entire product.

17. Second killer workflow

Executive Circular → Action

This is the feature that differentiates EduNexus from ordinary intranets.

Dean creates:

Circular: Preparation for Academic Accreditation Visit

Target:

All Department Heads

Quality Unit

Program Coordinators

The system asks:

Does this communication require action?

Yes.

Then:

Circular



Audience



Action Required



Department Tasks



Deadlines



Reminders



Escalation



Evidence



Completion Dashboard

The dean can therefore see:

Accreditation Preparation

Departments: 12

Completed: 8

In Progress: 3

Overdue: 1

Completion: 83%

Overdue:

Computer Science

Evidence package

Due: 10 Sep

Escalated: Yes

This is the core EduNexus value proposition.

18. Release dashboard KPIs

Executive

Open Requests

Pending Approvals

Overdue Tasks

SLA Breaches

Correspondence Pending

Department Completion

Workflow Cycle Time

Department Head

My Department

└─ Requests

└─ Approvals

└─ Tasks

└─ Overdue

└─ Correspondence

└─ Team workload

Employee

My Work

└─ Tasks

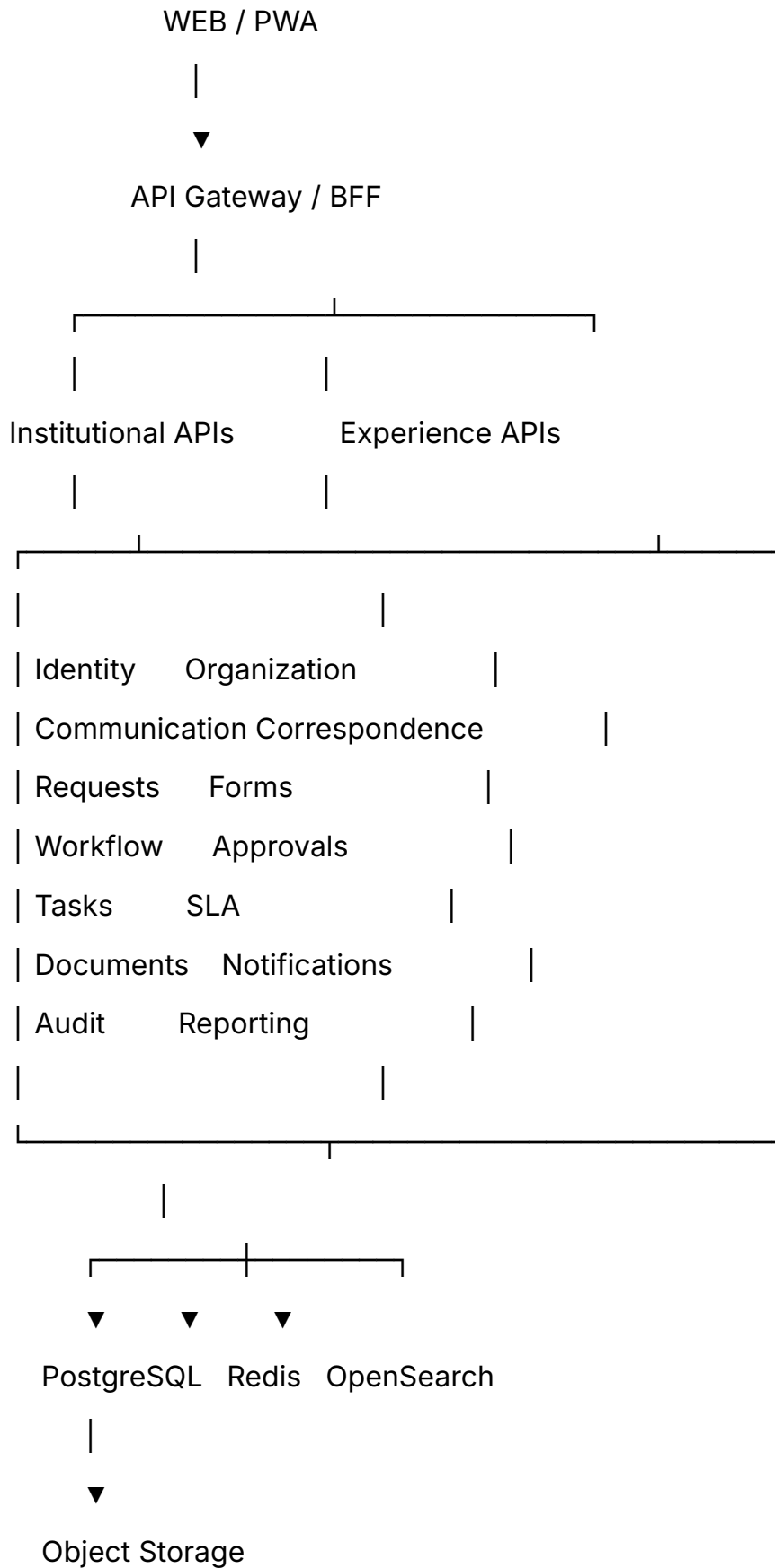
└─ Approvals

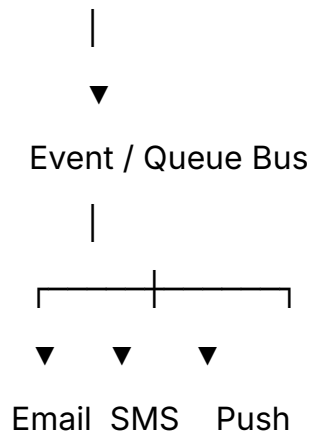
└─ Requests

└─ Messages

└─ Deadlines

19. Release 0.1 technical architecture





Recommended initial stack remains:

- **React + TypeScript / Next.js**
- **ASP.NET Core**
- **PostgreSQL**
- **Redis**
- **OpenSearch**
- **RabbitMQ**
- **S3-compatible storage**
- **OIDC/OAuth2**
- **Docker**
- **OpenTelemetry**
- **Prometheus/Grafana**

Kubernetes can be introduced for production environments where the deployment scale justifies it; it shouldn't become a prerequisite for the first developer installation.

20. Initial API surface

/auth

/users

/organizations

/people

/roles

/permissions

/communications

/announcements

/circulars

/correspondence

/inbox

/requests

/cases

/forms

/form-submissions

/workflows

/workflow-instances

/approvals

/tasks

/slas

/escalations

/documents

/templates

/notifications

/reports

/dashboards

/audit

/search

21. Initial database domains

Rather than creating one enormous database model, Release 0.1 uses bounded domains:

Identity

Organization

People

Communication

Correspondence

Requests

Forms

Workflow

Tasks

Documents

Notifications

Reporting

Audit

This leaves room for later domains:

Meetings

Committees

Decisions

Policies

Quality

Accreditation

Strategy

Risk

Resources

AI

22. Release 0.1 Definition of Done

The release is **not finished** when screens exist.

It is finished when an institution can perform this complete journey:

Create communication → target users → require action → generate work → route approval → assign task → enforce SLA → send notifications → collect evidence → verify completion → produce audit trail → report results.

That becomes the **minimum end-to-end product capability**.

23. What we build next

I recommend that the next engineering artifact be the actual:

EduNexus OS Release 0.1 Product Backlog

Not another conceptual document.

It should contain approximately:

- **12 modules**
- **60–80 epics/features**
- **250+ user stories**
- acceptance criteria
- business rules
- permissions
- API requirements
- database entities
- workflow definitions
- screen IDs

- reports
- integration requirements
- non-functional requirements
- release dependencies
- priority: P0/P1/P2
- development sequencing
- Definition of Done

And then we can turn that backlog into the **actual screen-by-screen product specification**, starting with:

Login → Home → My Work → Inbox → Correspondence → Request → Approval → Task → Documents → Reports → Administration.

That is the right next step if the goal is to move EduNexus from **blueprint → product → engineering execution**.

Today 7:21 PM

go for the next the new release and product upgrade

Absolutely. Let's move EduNexus from **Release 0.1 — Work Management Foundation** to the next major product upgrade.

EduNexus OS — Release 0.2

Institutional Governance & Decision Execution

Release 0.1 established:

Communication → Request → Workflow → Approval → Task → Evidence → Completion

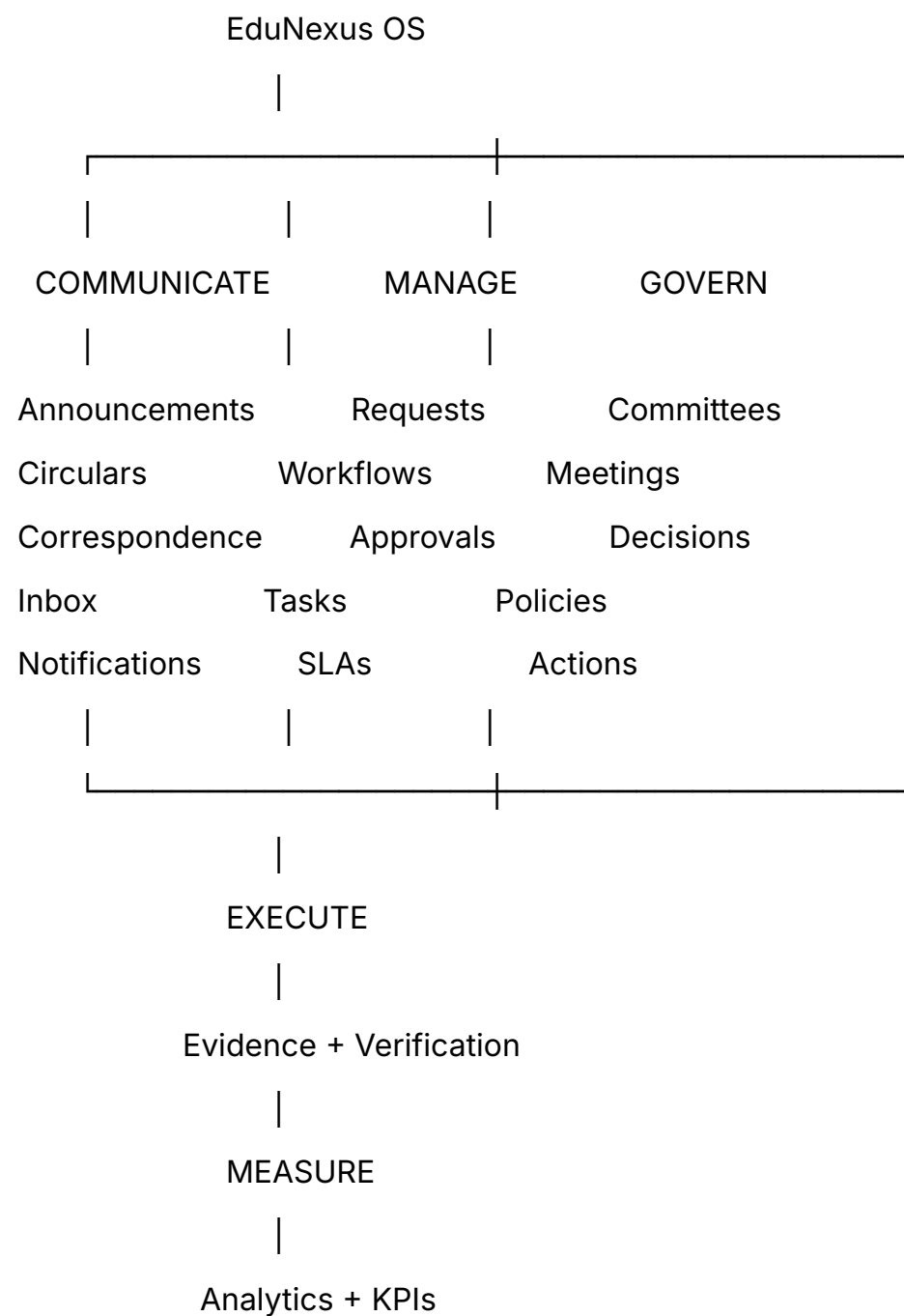
Release 0.2 adds the institutional governance layer:

Meeting → Deliberation → Decision → Assignment → Execution → Evidence → Verification → Institutional Knowledge

This is a significant product upgrade because EduNexus stops being primarily a workflow/correspondence product and becomes an **institutional management and governance platform**.

1. The Release 0.2 Product

The new product experience becomes:



2. New Product Positioning

Release 0.1:

Institutional Work Management

Release 0.2:

Institutional Management, Communication & Governance

Long-term:

EduNexus OS — The Digital Operating Layer for Educational Institutions

The product now covers the complete management loop:

Know → Decide → Assign → Execute → Verify → Learn

3. New Navigation

The main navigation should now become:

HOME

MY WORK

- └─ My Tasks
- └─ My Approvals
- └─ My Requests
- └─ Waiting For
- └─ Overdue

COMMUNICATION

- └─ Announcements
- └─ Circulars
- └─ Correspondence
- └─ Inbox
- └─ Notifications

WORK

- └─ Requests
- └─ Cases
- └─ Workflows

└─ Tasks

└─ SLAs

GOVERNANCE

└─ Committees

└─ Meetings

└─ Agendas

└─ Minutes

└─ Decisions

└─ Actions

POLICIES

└─ Policies

└─ Procedures

└─ Circulars

└─ Acknowledgements

└─ Review Calendar

DOCUMENTS

└─ Repository

└─ Templates

└─ Evidence

└─ Archive

PERFORMANCE

└─ Dashboards

└─ KPIs

- └─ Reports
- └─ Analytics

ADMINISTRATION

- └─ Organization
 - └─ Users
 - └─ Roles
 - └─ Forms
 - └─ Workflows
 - └─ Committees
 - └─ Templates
 - └─ Integrations
 - └─ Audit
-

4. Major New Module: Committees

This becomes one of EduNexus's signature capabilities.

Committee master

Committee

- └─ Committee ID
- └─ Name
- └─ Arabic Name
- └─ Type
- └─ Authority
- └─ Chair
- └─ Secretary
- └─ Members
- └─ Alternate Members

- └─ Term
- └─ Frequency
- └─ Quorum
- └─ Mandate
- └─ Reporting Line
- └─ Confidentiality
- └─ Status
- └─ Documents

Committee types:

- University Council
- Faculty Council
- Department Council
- Academic Committee
- Quality Committee
- Examination Committee
- Research Committee
- Ethics Committee
- Procurement Committee
- Recruitment Committee
- Student Affairs Committee
- Disciplinary Committee
- Temporary Committee
- Task Force

5. Committee Workspace

Each committee gets its own workspace.

Faculty Council	
Chair: Dean	
Overview	Meetings
Members	Decisions
Docs	
Next Meeting	
18 September 2026 — 10:00 AM	
Pending Actions: 17	
Overdue: 3	
Decisions this year: 42	
Implementation: 91%	

This is much stronger than merely having a meeting calendar.

6. Meeting Management

The meeting lifecycle becomes:

Create Meeting



Schedule



Invite Members



Collect Agenda Items



Collect Documents



Prepare Agenda



Approval



Send Agenda



Meeting



Attendance



Discussion



Minutes



Decisions



Actions



Follow-up

7. Agenda Builder

The secretary can create:

Meeting

|

└─ Opening

- └ Previous Minutes
- └ Follow-up Actions
- └ Agenda Item 1
 - | └ Background
 - | └ Proposal
 - | └ Documents
 - | └ Recommendation
- |
- └ Agenda Item 2
 - | └ Background
 - | └ Proposal
 - | └ Documents
- |
- └ Any Other Business

Each agenda item can produce a decision.

8. Digital Meeting Room

The meeting workspace:

FACULTY COUNCIL — 18 SEP 2026				
Agenda	Documents	Attendance	Chat	
03 — Curriculum Revision				
Background				

[document preview]	
Recommendation:	
Approve proposed curriculum revision.	
Discussion	
Decision	
[Approve] [Reject] [Defer] [Request Revision]	

9. Decision Management

This is arguably the most important upgrade.

A decision is no longer just text inside meeting minutes.

It becomes a **managed institutional object**.

Decision record

Decision Number

Title

Subject

Decision Type

Authority

Committee

Meeting

Agenda Item

Decision Date

Decision Text
Rationale
Owner
Affected Units
Deadline
Priority
Status
Related Policy
Related Documents
Actions
Evidence
Implementation %
Verification
Audit History

10. Decision Lifecycle

PROPOSED



DISCUSSION



APPROVED



PUBLISHED



ASSIGNED



IN EXECUTION



EVIDENCE SUBMITTED



VERIFICATION



IMPLEMENTED



CLOSED

Alternative:

APPROVED



REJECTED



RETURNED



REVISED



RECONSIDERED

11. The Decision-to-Execution Engine

This should become a major differentiator.

Example:

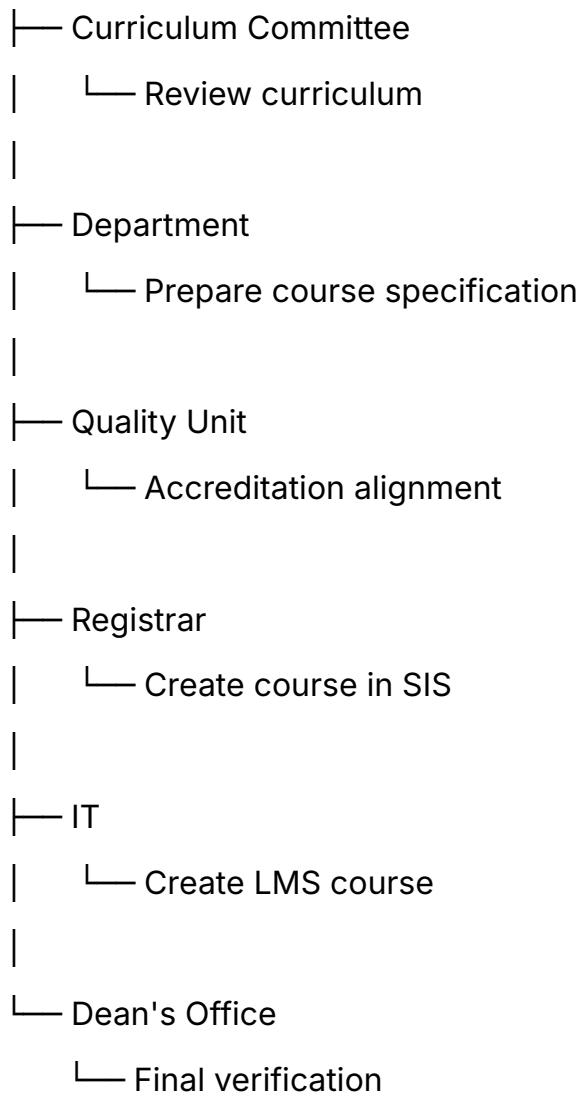
Faculty Council Decision

Establish a new cybersecurity course beginning Fall 2027.

EduNexus automatically generates:

Decision

|



Then the dean sees:

Decision D-2026-041

Implementation: 72%

- ✓ Curriculum approved
- ✓ Course specification completed
- ✓ Quality review completed
- SIS configuration
- LMS configuration

○ Final verification

This is the transition from:

meeting software

to

institutional execution software.

12. New Module: Policies & Procedures

Release 0.2 introduces institutional knowledge governance.

Policy lifecycle

Draft

↓

Review

↓

Legal / Quality Review

↓

Approval

↓

Publication

↓

Acknowledgement

↓

Implementation

↓

Periodic Review

↓

Revision / Retirement

Each policy has:

- Policy owner
 - Authority
 - Version
 - Effective date
 - Review date
 - Audience
 - Classification
 - Related procedures
 - Related forms
 - Related workflows
 - Acknowledgement requirement
 - Approval history
-

13. Policy-to-Workflow Relationship

This is important.

A policy should not merely sit in a PDF repository.

Example:

Procurement Policy

can be connected to:

Procurement Request



Procurement Workflow



Approval Matrix



Required Documents



Delegation Rules



Audit Rules

Therefore:

Policy → Procedure → Form → Workflow → Task → Evidence

This gives EduNexus a true governance model.

14. Policy Acknowledgement

For selected policies:

Policy Published



Audience Identified



Notification



User Opens Policy



Acknowledgement Required



Acknowledged



Timestamp + Version

Dashboard:

Policy:

Information Security Policy v3

Required:

1,240

Acknowledged:

1,172

Pending:

68

Completion:

94.5%

15. New Module: Institutional Calendar

Release 0.2 consolidates:

- meetings
- committee meetings
- deadlines
- policy reviews
- decision deadlines
- tasks
- events
- examinations through integration
- training
- institutional activities

into a unified calendar.

16. Governance Dashboard

The Dean/President/Director now gets:

INSTITUTIONAL GOVERNANCE			
Decisions	128		
Implemented	96		
In Progress	25		
Overdue	7		
Implementation Rate		87%	
Committee Performance			
Faculty Council	94%		
Quality Committee	89%		
Research Committee	82%		
Academic Committee	91%		
▲ Decisions Requiring Attention			
D-2026-031	Curriculum revision	12 days late	
D-2026-044	Lab renovation	5 days late	

17. New Executive KPI Model

Release 0.2 introduces:

Governance KPIs

- Decision implementation rate

- Decision cycle time
- Decision overdue rate
- Action completion rate
- Committee attendance
- Quorum compliance
- Meeting frequency
- Agenda turnaround time
- Minutes approval time
- Decision execution time

Policy KPIs

- Policy acknowledgement
- Policy review compliance
- Expired policies
- Policies pending approval
- Procedure adoption

Management KPIs

- Workflow cycle time
 - SLA compliance
 - Approval bottlenecks
 - Department workload
 - Escalations
 - Open cases
-

18. Release 0.2 AI Upgrade

We should introduce **controlled AI**, not unrestricted autonomous decision-making.

AI Meeting Assistant

Before meeting:

"Summarize pending actions from the previous three meetings."

During/after meeting:

"Generate draft minutes."

Then:

"Identify decisions."

Then:

"Extract actions and proposed owners."

The secretary reviews everything before publication.

19. AI Decision Assistant

A manager can ask:

"Show me all Faculty Council decisions that are overdue."

Or:

"Which departments have the highest implementation delays?"

Or:

"Summarize all decisions related to accreditation during 2026."

Or:

"Which policies require review within the next 30 days?"

The AI answers from authorized institutional data.

20. AI Governance Guardrail

The architecture must enforce:

AI

|

├— Retrieve authorized data

|

├— Analyze

|

└─ Generate recommendation

|

└─ Explain source

|

└─ Human approval

AI does **not** independently:

- approve institutional decisions
- change grades
- issue disciplinary decisions
- publish binding policies
- alter financial authority
- override workflow permissions

The human remains accountable.

21. New Data Model

Release 0.2 adds:

Committee

CommitteeMember

CommitteeTerm

Meeting

MeetingParticipant

Agenda

AgendaItem

AgendaDocument

Attendance

Minutes

MinutesVersion

Decision

DecisionAction

DecisionOwner

DecisionEvidence

DecisionVerification

Policy

PolicyVersion

PolicyApproval

PolicyAcknowledgement

Procedure

PolicyReview

CalendarEvent

InstitutionalEvent

Relationships:

Committee

|

└ Meeting

|

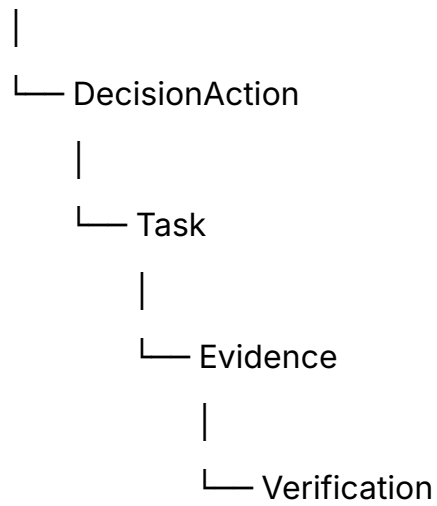
└ Agenda

|

└ Agendaltem

|

└ Decision



That relationship is fundamental to the product.

22. New API Domains

/committees

/committee-members

/meetings

/agendas

/agenda-items

/attendance

/minutes

/decisions

/decision-actions

/decision-evidence

/decision-verification

/policies

/policy-versions

/policy-approvals

/policy-acknowledgements

/procedures

/calendar

/events

/governance

/governance-dashboard

23. Release 0.2 Security Upgrade

Governance introduces more sensitive information.

Therefore:

Confidentiality levels

Public

Internal

Confidential

Restricted

Highly Confidential

And access becomes:

RBAC + ABAC + organizational scope + record classification.

Example:

A Faculty Council member may access:

Faculty Council

↓

Current term

↓

Committee documents

but not necessarily:

Recruitment Committee

Disciplinary Committee

Confidential HR cases

24. Governance Audit

Every critical governance operation is auditable:

Meeting Created

Meeting Changed

Member Added

Agenda Submitted

Agenda Approved

Document Added

Attendance Recorded

Minutes Drafted

Minutes Approved

Decision Proposed

Decision Approved

Decision Published

Action Assigned

Action Completed

Evidence Added

Decision Verified

Policy Published

Policy Acknowledged

Policy Revised

25. New Reports

Release 0.2 introduces:

Committee Reports

- Committee activity
- Member attendance
- Meeting statistics
- Quorum compliance
- Agenda statistics
- Minutes turnaround

Decision Reports

- Decisions by period
- Decisions by authority
- Decisions by department
- Decision implementation
- Overdue decisions
- Decision cycle time

Policy Reports

- Policy inventory
 - Policies by status
 - Pending approvals
 - Expired policies
 - Upcoming reviews
 - Acknowledgement compliance
-

26. Release 0.2 Signature Workflow

We should make this one of the product's demo workflows:

Faculty Council Decision

Secretary



Create Meeting



Collect Agenda



Chair Approves Agenda



Members Receive Agenda



Meeting



Attendance



Discussion



Decision



Chair Approval



Minutes



Decision Publication



Automatic Actions



Departments Execute



Evidence



Verification



Implementation Dashboard

This single workflow demonstrates the full governance capability.

27. Product Upgrade: "Institutional Memory"

This is a new strategic capability.

EduNexus should connect:

Communication



Correspondence



Meetings



Decisions



Policies



Documents



Tasks



Evidence

|

Reports

So someone can search:

"Show me everything related to the new postgraduate program."

and get:

Communications

Correspondence

Committee meetings

Agenda items

Decisions

Policies

Documents

Tasks

Evidence

Approvals

Reports

This becomes the institution's **institutional memory layer**.

28. Release 0.2 Product Differentiation

At this point EduNexus is no longer competing only with:

- correspondence systems
- workflow products
- intranets
- collaboration platforms

It begins occupying the space between:

Microsoft 365 / Teams + ERP + SIS + LMS + governance systems + workflow + institutional knowledge.

The strategic difference is:

EduNexus does not attempt to replace these systems. It orchestrates the institutional processes that run across them.

29. Release 0.2 Scope

Existing

12 core modules

Added

8 governance modules

13. Committees

14. Meetings

15. Agenda Management

16. Minutes

17. Decisions

18. Decision Execution

19. Policies

20. Procedures

Plus:

Governance Analytics

Institutional Calendar

AI Meeting Assistant

Institutional Memory

30. Release 0.2 Success Criteria

A production-ready Release 0.2 should allow an institution to execute this scenario entirely inside EduNexus:

Create committee → schedule meeting → build agenda → distribute papers → conduct meeting → record attendance → create minutes → approve minutes → issue decisions → automatically assign actions → track execution → collect evidence → verify implementation → report institutional performance.

If we can demonstrate that end-to-end, **Release 0.2 is a genuine product upgrade**, not simply additional screens.

31. The roadmap now becomes

RELEASE 0.1

Institutional Work Management

|



RELEASE 0.2

Governance & Decision Execution

|



RELEASE 0.3

Institutional Intelligence

|

├— AI Copilot

├— Advanced Analytics

├— Accreditation

├— Strategic Planning

├— Quality

└— Risk

|



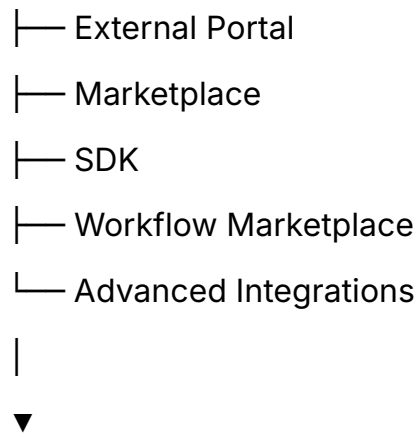
RELEASE 0.4

Institutional Ecosystem

|

├— Student Success

├— Resource Management



EDUNEXUS OS

Institutional Operating Layer

The strategic shift

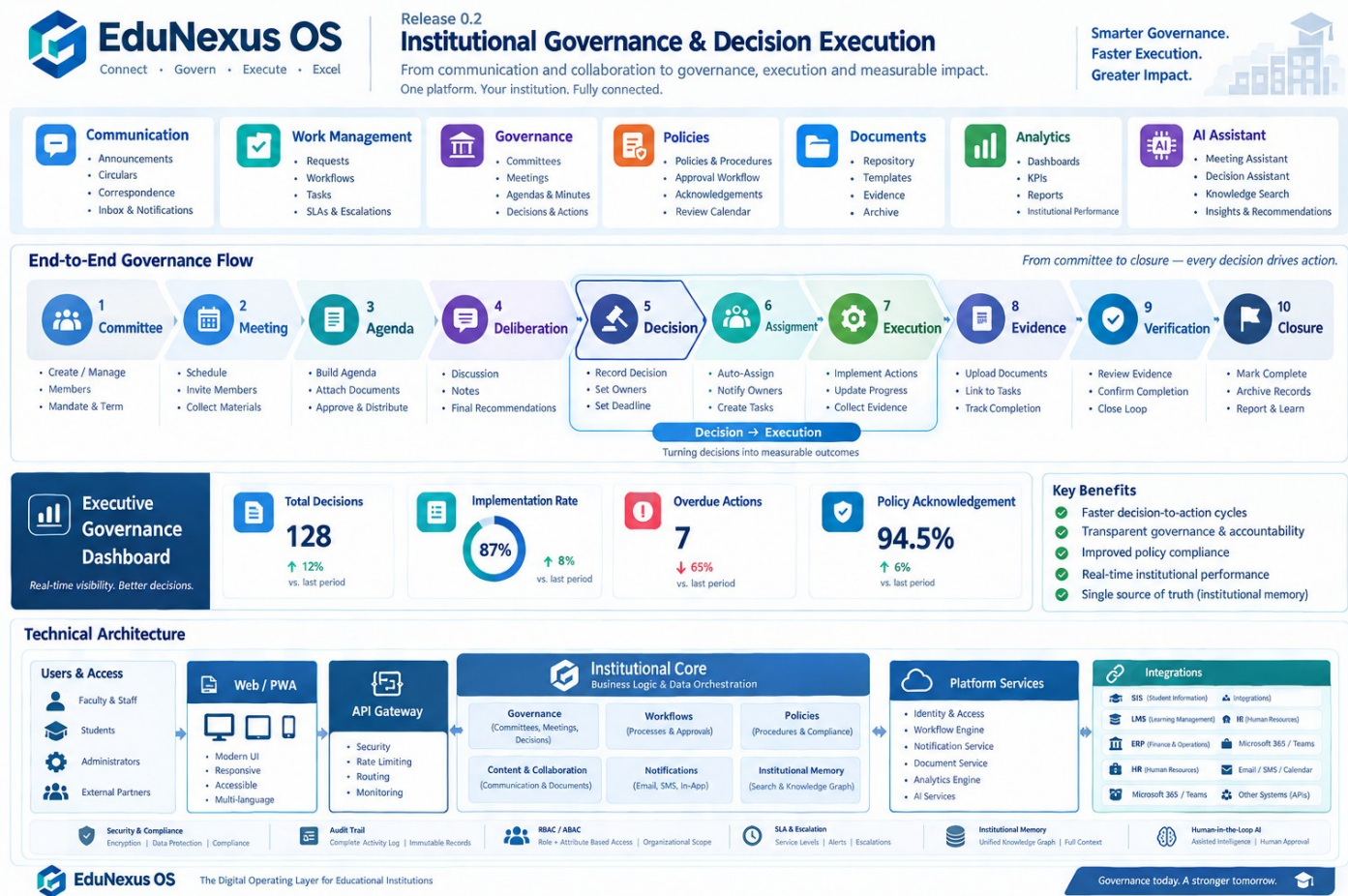
R0.1: *"Get the work done."*

R0.2: *"Make the institution governable."*

R0.3: *"Make the institution intelligent."*

R0.4: *"Make the institution an ecosystem."*

That gives us a much stronger product trajectory than continuously adding disconnected modules.



Qwen3.7-Plus

bbp

.docx

1.7 MB

check this product and propose the full technical document to build it

Thinking completed

EduNexus OS — Full Technical Build Document

Release 0.1 (Work Management Foundation) + Release 0.2 (Governance & Decision Execution)

TABLE OF CONTENTS

1. [Executive Summary](#)
2. [System Architecture](#)
3. [Technology Stack](#)
4. [Infrastructure & Deployment](#)
5. [Authentication & Authorization](#)
6. [Database Design — Bounded Domains](#)
7. [API Design & Specifications](#)
8. [Module Technical Specifications](#)
9. [Workflow Engine — Technical Design](#)
10. [Notification Engine — Technical Design](#)
11. [Document Management — Technical Design](#)
12. [Search & Institutional Memory](#)
13. [AI Integration Architecture](#)
14. [Security Architecture](#)
15. [Performance, Scalability & Caching](#)
16. [Monitoring & Observability](#)

- 17. [CI/CD & DevOps Pipeline](#)
- 18. [Testing Strategy](#)
- 19. [Data Migration & Multi-Tenancy](#)
- 20. [Release Phasing & Dependencies](#)
- 21. [Non-Functional Requirements](#)
- 22. [Definition of Done](#)

1. EXECUTIVE SUMMARY

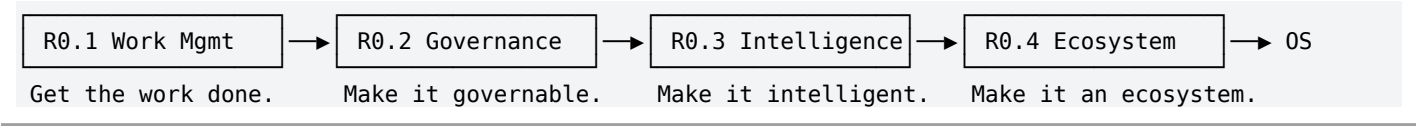
1.1 Product Vision

EduNexus OS is the **Digital Operating Layer for Educational Institutions**. It orchestrates institutional processes across communication, workflow, governance, and execution — without attempting to replace existing SIS, LMS, ERP, or collaboration systems.

1.2 Release Scope

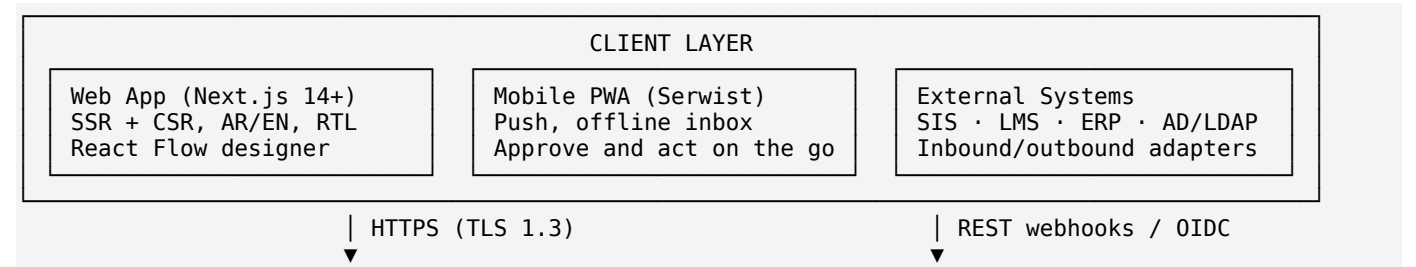
Release	Codename	Core Value
0.1	Work Management Foundation	Communication → Request → Workflow → Approval → Task → Evidence → Completion
0.2	Governance & Decision Execution	Meeting → Deliberation → Decision → Assignment → Execution → Evidence → Verification → Institutional Knowledge

1.3 Strategic Trajectory



2. SYSTEM ARCHITECTURE

2.1 High-Level Architecture Diagram



EDGE & IDENTITY

Ingress / WAF → Next.js BFF (route handlers)
TLS termination · rate limiting · CORS
Security headers · CSRF · HttpOnly session

Keycloak / Customer IdP
OIDC Auth Code + PKCE · MFA
Federation: AD/LDAP, Entra, SAML

| Bearer JWT (tenant_id + permission[] claims)

EDUNEXUS API – ASP.NET CORE 9 MODULAR MONOLITH (single process)

API surface: /api/* minimal-API endpoint groups · OpenAPI · /health · /metrics
Cross-cutting: JWT auth · HasPermission() · TenantScope (app.current_tenant → RLS)

R0.1 – WORK MANAGEMENT FOUNDATION

M01 Identity & Org tenants, roles	M02 Communication circulars, notices	M03 Correspondence register, route	M04 Unified Inbox one work queue
M05 Request Center catalog, cases	M06 Form Builder JSON schema	M07 Workflow Engine graph runner, nodes	M08 Approval Center decide, delegate
M09 Tasks & Actions assign, verify	M10 SLA/Escalation timers, breaches	M11 Documents versions, classes	M12 Audit append-only trail

R0.2 – GOVERNANCE & DECISION EXECUTION

M13 Committees charter, members	M14 Meetings agenda, minutes	M15 Decisions actions, evidence	M16 Policies publish, acknowledge
Calendar institutional	Gov. Dashboard KPIs, execution	AI Assistant minutes, guardrail	Inst. Memory search, lineage

SHARED KERNEL & BACKGROUND WORKERS (hosted services)

Domain events + outbox table	EventRelay outbox → RabbitMQ	SlaMonitor breach + escalate	NotificationConsumer + IntegrationDisp.
---------------------------------	---------------------------------	---------------------------------	--

EF Core 9 (Npgsql) · migrations job · OpenTelemetry SDK · Prometheus exporter



PostgreSQL 16 schema/domain RLS per tenant JSONB forms	Redis 7 cache, sessions SLA timers notif. dedup	OpenSearch 2.x institutional memory, AR/EN full-text	RabbitMQ 3.13 topic exchanges dead-letter event fan-out	MinIO / S3 documents, attachments, evidence files
---	--	---	--	--

| consumers



OBSERVABILITY

OpenTelemetry traces → Tempo (OTLP 4317)
Metrics → Prometheus → Grafana dashboards
Structured logs with trace ids
Alert rules: SLA breach, 5xx, queue depth

DELIVERY CHANNELS

In-app notifications (inbox)
Email · SMS · Push
Outbound webhooks (integrations)
Quiet hours + per-user preferences

Phase 2 (R0.3+): Workflow Engine, Notification Engine and AI Service are extracted from the monolith (strangler fig) behind an API gateway; each gets its own database.

2.2 Architectural Style

Phase 1 (R0.1–R0.2): Modular Monolith

- All modules deployed as a single ASP.NET Core process
- Each module is a separate project/namespace with explicit public API surface
- Inter-module communication via in-process events (MediatR) + RabbitMQ for async
- Database: single PostgreSQL instance with schema-per-module isolation

Phase 2 (R0.3+): Strangler Fig → Microservices

- Extract high-load modules (Workflow Engine, Notification Engine, AI Service) into independent services
- Each service gets its own database
- API Gateway handles routing

2.3 Key Architectural Decisions

Decision	Choice	Rationale
Architecture style	Modular Monolith first	Faster initial delivery; avoids distributed system complexity for <1000 concurrent users
Frontend framework	Next.js 14+ (App Router)	SSR for initial load, CSR for interactive dashboards, PWA support built-in
Backend framework	ASP.NET Core 8/9	Enterprise-grade, high performance, strong typing, excellent DI
Database	PostgreSQL 16	JSONB for dynamic forms, full-text search fallback, row-level security for multi-tenancy
Cache	Redis 7	Session, workflow state, SLA timers, notification dedup
Search	OpenSearch 2.x	Institutional memory, full-text across all entities, Arabic support
Message Queue	RabbitMQ 3.13	Reliable delivery, dead-letter queues, topic exchanges for event fan-out
Storage	S3-compatible (MinIO / AWS S3)	Documents, attachments, evidence files
Auth	OIDC/OAuth2 (Keycloak or Duende)	Standards-based, SSO, LDAP/AD federation
Containerization	Docker + Docker	Simple developer onboarding

Decision	Choice	Rationale
	Compose (dev)	
Orchestration	Kubernetes (production only)	When scale justifies it

3. TECHNOLOGY STACK

3.1 Frontend

Layer	Technology	Version
Framework	Next.js (App Router)	14+
Language	TypeScript	5.x
UI Library	shadcn/ui + Radix Primitives	Latest
State Management	Zustand + React Query (TanStack)	Latest
Form Handling	React Hook Form + Zod	Latest
i18n	next-intl	Latest
RTL Support	Tailwind CSS rtl: variants	—
Charts	Recharts / Nivo	Latest
PWA	next-pwa / Serwist	Latest
Rich Text	TipTap	Latest
Workflow Designer	React Flow	Latest
Calendar	FullCalendar	Latest
PDF Viewer	react-pdf	Latest

3.2 Backend

Layer	Technology	Version
Runtime	ASP.NET Core	8/9
Language	C#	12+

Layer	Technology	Version
API Style	REST + minimal APIs + Controllers	—
ORM	Entity Framework Core	8+
Validation	FluentValidation	Latest
Mediator	MediatR	Latest
Mapping	Mapster	Latest
Background Jobs	Hangfire (or Quartz.NET)	Latest
Workflow Engine	Custom (state machine + Elsa Core evaluation)	—
Real-time	SignalR	Built-in
Health Checks	AspNetCore.HealthChecks	Latest

3.3 Data & Infrastructure

Layer	Technology	Version
Primary DB	PostgreSQL	16
Cache	Redis	7
Search	OpenSearch	2.x
Queue	RabbitMQ	3.13
Object Storage	MinIO (dev) / AWS S3 (prod)	Latest
Auth Server	Keycloak (self-hosted) or Duende IdentityServer	Latest
Monitoring	Prometheus + Grafana	Latest
Tracing	OpenTelemetry + Jaeger	Latest
Logging	Serilog → Seq / ELK	Latest
Container	Docker	Latest

Layer	Technology	Version
Orchestration	Kubernetes (prod)	1.28+
CI/CD	GitHub Actions / GitLab CI	—

4. INFRASTRUCTURE & DEPLOYMENT

4.1 Environment Strategy

Environment	Purpose	Infrastructure
Local	Developer workstation	Docker Compose (all dependencies)
Dev	Integration testing	Single VM / small K8s cluster
Staging	Pre-production validation	Mirrors production
Production	Live tenants	K8s cluster (managed)

4.2 Docker Compose (Developer Setup)

yaml

```
# infra/docker-compose.yml (core services; observability stack omitted here)
services:
  postgres:
    image: postgres:16
    container_name: edunexus-postgres
    environment:
      POSTGRES_DB: edunexus
      POSTGRES_USER: edunexus
      POSTGRES_PASSWORD: edunexus
    ports: ["5433:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U edunexus"]
      interval: 5s
      retries: 10
  redis:
    image: redis:7
    container_name: edunexus-redis
    command: redis-server --appendonly yes
    ports: ["6380:6379"]
    volumes: [redisdata:/data]
  rabbitmq:
    image: rabbitmq:3-management
    container_name: edunexus-rabbitmq
    ports: ["5673:5672", "15673:15672"]
  minio:
    image: minio/minio
    container_name: edunexus-minio
    command: server /data --console-address ":9001"
```

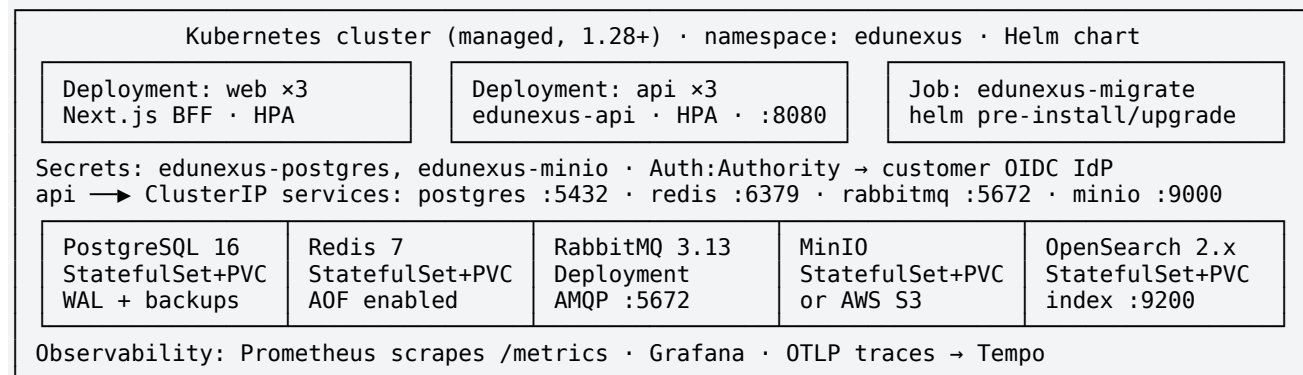
```

environment:
  MINIO_ROOT_USER: minio
  MINIO_ROOT_PASSWORD: minio12345
  ports: ["9002:9000", "9003:9001"]
  volumes: [miniodata:/data]
opensearch:
  image: opensearchproject/opensearch:2
  container_name: edunexus-opensearch
  environment:
    discovery.type: single-node
    DISABLE_SECURITY_PLUGIN: "true"
    OPENSEARCH_JAVA_OPTS: "-Xms512m -Xmx512m"
  ports: ["9201:9200"]
  volumes: [osdata:/usr/share/opensearch/data]
keycloak: # opt-in local OIDC IdP: docker compose --profile idp up keycloak
  image: quay.io/keycloak/keycloak:26.0
  profiles: [idp]
  command: start-dev
  ports: ["8080:8080"]
volumes: { pgdata: {}, redisdata: {}, miniodata: {}, osdata: {} }

```

4.3 Production Architecture (Kubernetes)

Users → HTTPS → WAF / Load Balancer → Ingress (TLS 1.3) / → web · /api → api



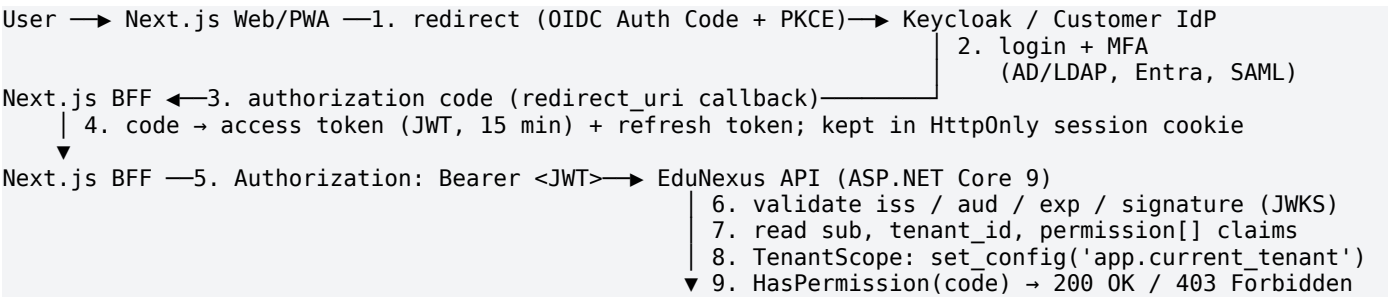
4.4 Resource Estimates (Initial Production)

Component	CPU	RAM	Storage
API (3 pods)	2 cores each	4 GB each	—
Web (3 pods)	1 core each	2 GB each	—
PostgreSQL	4 cores	16 GB	500 GB SSD
Redis	2 cores	8 GB	—
RabbitMQ	2 cores	4 GB	50 GB
OpenSearch	4 cores	16 GB	200 GB

Component	CPU	RAM	Storage
MinIO/S3	2 cores	4 GB	1 TB+

5. AUTHENTICATION & AUTHORIZATION

5.1 Authentication Flow



5.2 Token Structure

json

```
{
  "iss": "https://idp.local/realms/edunexus",
  "sub": "8f14e45f-ceeda-467a-9575-1f2a3b4c5d6e",
  "aud": "edunexus-api",
  "iat": 1791273600,
  "exp": 1791274500,
  "tenant_id": "3c9d2b1a-7e4f-4a6b-9c8d-0e1f2a3b4c5d",
  "permission": ["request:create", "approval:read", "approval:decide", "task:read",
    "task:update", "meeting:read", "decision:read", "policy:ack"]
}
```

5.3 Authorization Model

Hybrid RBAC + ABAC + Organizational Scope

1

Layer	Mechanism	Example
RBAC	Role → Permission mapping	Dean → approval:approve
ABAC	Attribute-based policies	classification <= user.clearance
Org Scope	Hierarchical data filtering	Dean sees only their faculty
Delegation	Temporary authority transfer	Department head delegates to deputy
Classification	Record-level confidentiality	Restricted documents need special role

5.4 Permission Catalog (R0.1 + R0.2)

R0.1 – WORK MANAGEMENT FOUNDATION

Platform	tenant:read, tenant:create
Organization	org:read, org:create
People	person:read, person:create
Roles	role:read, role:create, role:assign
Correspondence	correspondence:read, correspondence:create, correspondence:confidential
Forms	form:read, form:manage
Requests	request:read, request:create
Workflow	workflow:read, workflow:create, workflow:manage
Approvals	approval:read, approval:decide
Tasks	task:read, task:update
SLA/Escalation	escalation:* (any code with this prefix marks the escalation owner role)
Documents	document:read, document:create, document:update, document:share, document:write, document:manage
Notifications	notification:read, notification:manage
Inbox / Audit	inbox:read, audit:read

R0.2 – GOVERNANCE & DECISION EXECUTION

Committees	committee:read, committee:create
Meetings	meeting:read, meeting:create, meeting:update
Decisions	decision:read, decision:create, action:update
Policies	policy:read, policy:create, policy:ack
Search / AI	search:read, ai:read, ai:write, ai:manage, ai:admin
Integrations	integration:write, integration:manage, integration:admin

Format <resource>:<action>; roles bundle codes; tokens carry them as permission claims.

5.5 Delegation Engine

csharp

```
public sealed record AuthorityDelegation(Guid Id, Guid TenantId, Guid FromPersonId,
    Guid ToPersonId, string Scope, DateTimeOffset ExpiresAt);
private static async Task<Guid> ApplyDelegationFallbackAsync(AppDbContext db, Guid tenantId,
    Guid targetPersonId, CancellationToken ct)
{
    var now = DateTimeOffset.UtcNow;
    var delegation = await db.AuthorityDelegations
        .Where(d => d.TenantId == tenantId && d.FromPersonId == targetPersonId && d.ExpiresAt > now)
        .OrderByDescending(d => d.ExpiresAt).FirstOrDefaultAsync(ct);
    return delegation?.ToPersonId is { } to && to != Guid.Empty ? to : targetPersonId;
}
```

6. DATABASE DESIGN — BOUNDED DOMAINS

6.1 Schema Strategy

Each bounded domain gets its own PostgreSQL schema within a single database:

sql

```
-- One database (edunexus); one schema per bounded domain. Tables carry tenant_id + RLS.
CREATE SCHEMA IF NOT EXISTS identity;
CREATE SCHEMA IF NOT EXISTS organization;
CREATE SCHEMA IF NOT EXISTS people;
CREATE SCHEMA IF NOT EXISTS communication;
CREATE SCHEMA IF NOT EXISTS correspondence;
CREATE SCHEMA IF NOT EXISTS requests;
CREATE SCHEMA IF NOT EXISTS forms;
CREATE SCHEMA IF NOT EXISTS workflow;
CREATE SCHEMA IF NOT EXISTS tasks;
CREATE SCHEMA IF NOT EXISTS documents;
CREATE SCHEMA IF NOT EXISTS notifications;
```

```

CREATE SCHEMA IF NOT EXISTS audit;
CREATE SCHEMA IF NOT EXISTS committees; -- R0.2
CREATE SCHEMA IF NOT EXISTS meetings; -- R0.2
CREATE SCHEMA IF NOT EXISTS decisions; -- R0.2
CREATE SCHEMA IF NOT EXISTS policies; -- R0.2
CREATE SCHEMA IF NOT EXISTS calendar; -- R0.2

```

6.2 Domain: Identity (identity.)

sql

```

-- Platform tenant registry (not RLS-scoped; read by the API at token validation)
CREATE TABLE identity.tenants (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  slug        varchar(63) NOT NULL UNIQUE CHECK (slug ~ '^[a-z0-9-]{3,63}$'),
  name        varchar(200) NOT NULL,
  is_active   boolean      NOT NULL DEFAULT true,
  created_at  timestampz   NOT NULL DEFAULT now()
);

-- Login account; 1:1 with people.persons (directory record, section 6.4)
CREATE TABLE identity.users (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid          NOT NULL REFERENCES identity.tenants(id),
  person_id   uuid          NOT NULL, -- → people.persons(id)
  external_sub varchar(200) NOT NULL, -- OIDC "sub" from Keycloak
  username    varchar(150) NOT NULL,
  email       varchar(250),
  preferred_lang char(2) NOT NULL DEFAULT 'ar' CHECK (preferred_lang IN ('ar','en')),
  mfa_enabled boolean      NOT NULL DEFAULT false,
  status      varchar(20) NOT NULL DEFAULT 'Active'
  CHECK (status IN ('Invited','Active','Locked','Disabled')),
  last_login_at timestampz,
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, external_sub),
  UNIQUE (tenant_id, username)
);

CREATE TABLE identity.roles (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid          NOT NULL REFERENCES identity.tenants(id),
  code        varchar(50) NOT NULL, -- e.g. DEAN, DEPT_HEAD
  name        varchar(200) NOT NULL,
  permissions jsonb        NOT NULL DEFAULT '[]', -- ["approval:decide","task:read"]
  UNIQUE (tenant_id, code)
);

CREATE TABLE identity.role_assignments (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid          NOT NULL REFERENCES identity.tenants(id),
  person_id   uuid          NOT NULL, -- → people.persons(id)
  role_id     uuid          NOT NULL REFERENCES identity.roles(id),
  scope       varchar(200) NOT NULL DEFAULT '*', -- org scope: '*' or unit path
  expires_at  timestampz,
  created_at  timestampz NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, person_id, role_id)
);

-- Temporary authority transfer, checked before an approval is presented (8.8)
CREATE TABLE identity.delegations (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid          NOT NULL REFERENCES identity.tenants(id),
  from_person_id uuid      NOT NULL,
  to_person_id uuid      NOT NULL CHECK (to_person_id <> from_person_id),
  scope       varchar(200) NOT NULL, -- e.g. 'approval:decide'

```

```

    starts_at          timestampz NOT NULL DEFAULT now(),
    expires_at         timestampz NOT NULL,
    revoked_at         timestampz,
    CHECK (expires_at > starts_at)
);
CREATE INDEX ix_delegations_active ON identity.delegations (tenant_id, to_person_id, expires_at)
WHERE revoked_at IS NULL;

-- Per-tenant counters for CORR-2026-000123, REQ-2026-001245, ... (row-locked upsert)
CREATE TABLE identity.tenant_sequences (
    tenant_id          uuid          NOT NULL REFERENCES identity.tenants(id),
    scope              varchar(50)   NOT NULL,
    next_value         bigint        NOT NULL DEFAULT 1,
    PRIMARY KEY (tenant_id, scope)
);

```

6.3 Domain: Organization (organization.)

sql

```

CREATE EXTENSION IF NOT EXISTS ltree;

CREATE TABLE organization.units (
    id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id         uuid          NOT NULL REFERENCES identity.tenants(id),
    parent_id         uuid          REFERENCES organization.units(id),
    unit_type         varchar(20)   NOT NULL CHECK (unit_type IN ('University','Campus',
    'Faculty','Department','Program','Unit','Center','Institute')),
    code              varchar(50)   NOT NULL,
    name              varchar(200)  NOT NULL,
    name_ar           varchar(200),
    path              ltree         NOT NULL,          -- e.g. uni.main.eng.cs
    leader_person_id  uuid,          -- → people.persons(id)
    deputy_person_id  uuid,
    is_active         boolean       NOT NULL DEFAULT true,
    UNIQUE (tenant_id, code)
);
CREATE INDEX ix_units_path ON organization.units USING gist (path);

```

6.4 Domain: People (people.)

sql

```

CREATE TABLE people.persons (
    id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id         uuid          NOT NULL REFERENCES identity.tenants(id),
    person_type       varchar(20)   NOT NULL CHECK (person_type IN ('Employee','Student','Other')),
    employee_no       varchar(50),   -- or student number (SIS/HRIS key)
    full_name         varchar(250)  NOT NULL,
    full_name_ar      varchar(250),
    email             varchar(250),
    department_id     uuid          REFERENCES organization.units(id),
    position_title    varchar(150),
    academic_rank     varchar(50),   -- Professor, Lecturer, ...
    manager_id        uuid          REFERENCES people.persons(id),
    is_active         boolean       NOT NULL DEFAULT true,
    created_at        timestampz    NOT NULL DEFAULT now(),
    updated_at        timestampz    NOT NULL DEFAULT now(),
    deleted_at        timestampz,    -- soft delete for directory entities
    UNIQUE (tenant_id, employee_no)
);
CREATE INDEX ix_persons_dept ON people.persons (tenant_id, department_id) WHERE deleted_at IS NULL;
CREATE INDEX ix_persons_email ON people.persons (tenant_id, lower(email));

```

6.5 Domain: Communication (communication.)

sql

```
-- Announcement, Circular and Directive share one table; kind drives behaviour
CREATE TABLE communication.announcements (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id         uuid          NOT NULL REFERENCES identity.tenants(id),
  kind              varchar(20)   NOT NULL CHECK (kind IN ('Announcement','Circular','Directive')),
  reference_no      varchar(30),  -- circulars only, e.g. CIR-2026-0042
  title             varchar(300)  NOT NULL,
  body              text          NOT NULL,      -- TipTap HTML, Arabic RTL supported
  author_id         uuid          NOT NULL REFERENCES people.persons(id),
  issuing_unit_id   uuid          REFERENCES organization.units(id),
  priority          varchar(20)   NOT NULL DEFAULT 'Normal'
  CHECK (priority IN ('FYI','Normal','Important','Urgent','Emergency')),
  confidentiality   varchar(20)  NOT NULL DEFAULT 'Internal'
  CHECK (confidentiality IN ('Public','Internal','Confidential',
                             'Restricted','HighlyConfidential')),
  requires_action   boolean       NOT NULL DEFAULT false, -- true → DirectivePublished event
  due_at            timestamptz,
  status            varchar(20)   NOT NULL DEFAULT 'Draft'
  CHECK (status IN ('Draft','Published')),
  created_at        timestamptz  NOT NULL DEFAULT now(),
  published_at      timestamptz,
  CHECK (requires_action = false OR due_at IS NOT NULL)
);
CREATE INDEX ix_announcements_kind ON communication.announcements (tenant_id, kind, status);

-- Targeting rules evaluated by the audience resolver (org hierarchy + roles)
CREATE TABLE communication.audiences (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id         uuid          NOT NULL REFERENCES identity.tenants(id),
  communication_id   uuid          NOT NULL REFERENCES communication.announcements(id)
  ON DELETE CASCADE,
  unit_id           uuid          REFERENCES organization.units(id),
  include_children   boolean       NOT NULL DEFAULT true,
  role_code          varchar(50),  -- e.g. DEPT_HEAD
  person_id         uuid          REFERENCES people.persons(id),
  CHECK (num_nonnulls(unit_id, role_code, person_id) >= 1)
);

-- Resolved recipients: one row per person, written at publish time
CREATE TABLE communication.recipients (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id         uuid          NOT NULL REFERENCES identity.tenants(id),
  communication_id   uuid          NOT NULL REFERENCES communication.announcements(id)
  ON DELETE CASCADE,
  person_id         uuid          NOT NULL REFERENCES people.persons(id),
  task_id           uuid,         -- → tasks.items for directives
  read_at           timestamptz,
  acknowledged_at   timestamptz,
  UNIQUE (tenant_id, communication_id, person_id)
);
CREATE INDEX ix_recipients_unread ON communication.recipients (tenant_id, person_id)
WHERE read_at IS NULL;
```

6.6 Domain: Correspondence (correspondence.)

sql

```
CREATE TABLE correspondence.records (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id         uuid          NOT NULL REFERENCES identity.tenants(id),
  number            varchar(30)   NOT NULL,      -- CORR-2026-{sequence}
  type              varchar(20)   NOT NULL CHECK (type IN ('Incoming','Outgoing','Internal')),
  subject           varchar(300)  NOT NULL,
  content           text          NOT NULL,
```

```

source          varchar(300),          -- external sender / issuing body
author_id       uuid          NOT NULL REFERENCES people.persons(id),
priority        varchar(20)    NOT NULL DEFAULT 'Normal'
                CHECK (priority IN ('Normal','High','Urgent')),
classification   varchar(100),          -- subject classification code
is_confidential boolean        NOT NULL DEFAULT false,
status          varchar(20)    NOT NULL DEFAULT 'Received' CHECK (status IN ('Received',
                'Registered','Classified','Assigned','Action','FollowUp','Response',
                'Closed')),
due_date        date,
assigned_to     uuid          REFERENCES people.persons(id),
department_id   uuid          REFERENCES organization.units(id),
parent_id       uuid          REFERENCES correspondence.records(id), -- reply thread
related_request_id uuid,          -- → requests.cases(id)
related_task_id uuid,          -- → tasks.items(id)
created_at      timestamptz NOT NULL DEFAULT now(),
                UNIQUE (tenant_id, number)
);
CREATE INDEX ix_corr_status ON correspondence.records (tenant_id, status, due_date);

-- From / To / CC parties, internal persons or external bodies
CREATE TABLE correspondence.correspondents (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id      uuid          NOT NULL REFERENCES identity.tenants(id),
  correspondence_id uuid        NOT NULL REFERENCES correspondence.records(id),
  party_role     varchar(10)   NOT NULL CHECK (party_role IN ('From','To','CC')),
  person_id      uuid          REFERENCES people.persons(id),
  display_name   varchar(250)  NOT NULL,
  is_external    boolean       NOT NULL DEFAULT false
);

```

6.7 Domain: Requests (requests.)

sql

```

CREATE TABLE requests.cases (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id      uuid          NOT NULL REFERENCES identity.tenants(id),
  number        varchar(30)    NOT NULL,          -- REQ-2026-001245
  category       varchar(20)    NOT NULL CHECK (category IN ('Academic','Administrative',
  'HR','Finance','Procurement','IT','Facilities','StudentAffairs',
  'Research','Quality','Other')),
  title         varchar(300)   NOT NULL,
  form_id       uuid,          -- → forms.definitions(id)
  submitter_id  uuid          NOT NULL REFERENCES people.persons(id),
  department_id  uuid          REFERENCES organization.units(id),
  status        varchar(20)    NOT NULL DEFAULT 'Draft' CHECK (status IN ('Draft',
  'Submitted','InReview','ChangesRequested','Approved','Rejected',
  'Closed')),
  workflow_instance_id uuid,          -- → workflow.instances(id)
  created_at    timestamptz NOT NULL DEFAULT now(),
  submitted_at  timestamptz,
  closed_at     timestamptz,
  UNIQUE (tenant_id, number)
);
CREATE INDEX ix_cases_status  ON requests.cases (tenant_id, status);
CREATE INDEX ix_cases_category ON requests.cases (tenant_id, category);

```

6.8 Domain: Forms (forms.)

sql

```

CREATE TABLE forms.definitions (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id      uuid          NOT NULL REFERENCES identity.tenants(id),
  code          varchar(50)    NOT NULL,          -- e.g. PURCHASE_REQUEST
  name          varchar(200)   NOT NULL,

```

```

    category          varchar(20) NOT NULL,      -- same values as requests.cases
    schema             jsonb       NOT NULL,      -- [{key,label,type,required,...}]
    version            int         NOT NULL DEFAULT 1,
    workflow_definition_id uuid,          -- default route for the category
    is_active          boolean     NOT NULL DEFAULT true,
    UNIQUE (tenant_id, code)
);

CREATE TABLE forms.versions (
    tenant_id          uuid         NOT NULL REFERENCES identity.tenants(id),
    form_id            uuid         NOT NULL REFERENCES forms.definitions(id),
    version            int          NOT NULL,
    schema             jsonb        NOT NULL,
    published_at       timestampz   NOT NULL DEFAULT now(), -- each publish = new version
    PRIMARY KEY (form_id, version)
);

CREATE TABLE forms.submissions (
    id                 uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id          uuid         NOT NULL REFERENCES identity.tenants(id),
    request_id         uuid         NOT NULL REFERENCES requests.cases(id),
    form_id            uuid         NOT NULL,
    form_version       int          NOT NULL,
    data               jsonb        NOT NULL,
    submitted_by       uuid         NOT NULL REFERENCES people.persons(id),
    submitted_at       timestampz   NOT NULL DEFAULT now(),
    FOREIGN KEY (form_id, form_version) REFERENCES forms.versions(form_id, version)
);
CREATE INDEX ix_submissions_data ON forms.submissions USING gin (data jsonb_path_ops);

```

6.9 Domain: Workflow (workflow.)

sql

```

CREATE TABLE workflow.definitions (
    id                 uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id          uuid         NOT NULL REFERENCES identity.tenants(id),
    code               varchar(50)  NOT NULL,
    name               varchar(200) NOT NULL,
    version            int          NOT NULL DEFAULT 1,
    nodes              jsonb        NOT NULL,      -- nodes, edges, slaDays per node
    escalation          jsonb        NOT NULL DEFAULT '[]', -- escalation chain (9.4)
    is_active          boolean     NOT NULL DEFAULT false,
    UNIQUE (tenant_id, code)
);

CREATE TABLE workflow.instances (
    id                 uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id          uuid         NOT NULL REFERENCES identity.tenants(id),
    definition_id       uuid         NOT NULL REFERENCES workflow.definitions(id),
    entity_type         varchar(50) NOT NULL,      -- 'Request', 'Correspondence', ...
    entity_id          uuid         NOT NULL,
    current_node_id     varchar(50) NOT NULL,
    status              varchar(20) NOT NULL DEFAULT 'Running'
    CHECK (status IN ('Running', 'Completed', 'Closed', 'Rejected')),
    context             jsonb        NOT NULL DEFAULT '{}',
    updated_at         timestampz   NOT NULL DEFAULT now()
);
CREATE INDEX ix_instances_entity ON workflow.instances (tenant_id, entity_type, entity_id);

CREATE TABLE workflow.approvals (
    id                 uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id          uuid         NOT NULL REFERENCES identity.tenants(id),
    instance_id        uuid         REFERENCES workflow.instances(id),
    node_id            varchar(50),
    entity_type         varchar(50) NOT NULL,

```

```

entity_id      uuid      NOT NULL,
assignee_id    uuid      NOT NULL REFERENCES people.persons(id),
delegation_id  uuid      REFERENCES identity.delegations(id),
status         varchar(20) NOT NULL DEFAULT 'Pending' CHECK (status IN
('Pending', 'Approved', 'Rejected', 'Cancelled', 'ChangesRequested')),
priority       varchar(20) NOT NULL DEFAULT 'Normal', -- Normal | Accelerated
due_at         timestampz NOT NULL,
decided_at     timestampz,
decided_by     uuid      REFERENCES people.persons(id),
comment        text
);
CREATE INDEX ix_approvals_inbox ON workflow.approvals (tenant_id, assignee_id, status);

```

6.10 Domain: Tasks (tasks.)

sql

```

CREATE TABLE tasks.items (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid      NOT NULL REFERENCES identity.tenants(id),
  title       varchar(300) NOT NULL,
  description  varchar(2000) NOT NULL DEFAULT '',
  source_type  varchar(20) NOT NULL DEFAULT 'Manual'
  CHECK (source_type IN ('Workflow', 'Decision', 'Directive', 'Manual')),
  source_id   uuid, -- instance, decision or circular id
  approval_id uuid      REFERENCES workflow.approvals(id),
  assignee_id uuid      NOT NULL REFERENCES people.persons(id),
  department_id uuid     REFERENCES organization.units(id),
  priority    varchar(20) NOT NULL DEFAULT 'Normal'
  CHECK (priority IN ('Low', 'Normal', 'High', 'Urgent')),
  status      varchar(20) NOT NULL DEFAULT 'New' CHECK (status IN ('New', 'Assigned',
  'InProgress', 'Waiting', 'Blocked', 'Completed', 'Verified', 'Cancelled', 'Overdue')),
  progress    smallint NOT NULL DEFAULT 0 CHECK (progress BETWEEN 0 AND 100),
  start_date  date,
  due_at      timestampz NOT NULL,
  verified_by uuid      REFERENCES people.persons(id),
  verified_at timestampz,
  created_at  timestampz NOT NULL DEFAULT now()
);
CREATE INDEX ix_items_assignee ON tasks.items (tenant_id, assignee_id, status);
CREATE INDEX ix_items_due      ON tasks.items (tenant_id, due_at)
  WHERE status NOT IN ('Completed', 'Verified', 'Cancelled');

CREATE TABLE tasks.dependencies (
  tenant_id   uuid NOT NULL REFERENCES identity.tenants(id),
  task_id     uuid NOT NULL REFERENCES tasks.items(id) ON DELETE CASCADE,
  depends_on_id uuid NOT NULL REFERENCES tasks.items(id) ON DELETE CASCADE,
  PRIMARY KEY (task_id, depends_on_id),
  CHECK (task_id <> depends_on_id)
);

CREATE TABLE tasks.evidence (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid      NOT NULL REFERENCES identity.tenants(id),
  task_id     uuid      NOT NULL REFERENCES tasks.items(id),
  uploaded_by uuid      NOT NULL REFERENCES people.persons(id),
  object_key  varchar(500) NOT NULL, -- S3/MinIO key
  file_name   varchar(255) NOT NULL,
  note        text,
  at          timestampz NOT NULL DEFAULT now()
);

```

6.11 Domain: Documents (documents.)

sql

```

CREATE TABLE documents.records (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  workspace_id UUID,                                -- documents.workspaces (DMS folders)
  title       VARCHAR(300) NOT NULL,
  status      VARCHAR(20) NOT NULL DEFAULT 'Draft'
              CHECK (status IN ('Draft','Published','Archived')),
  classification VARCHAR(20) NOT NULL DEFAULT 'Internal'
              CHECK (classification IN ('Public','Internal','Confidential','Restricted')),
  current_version INT NOT NULL DEFAULT 1,
  entity_type   VARCHAR(50),                        -- attached transaction, e.g. 'Request'
  entity_id     UUID,
  extracted_text TEXT,                               -- OCR/text extraction for OpenSearch
  retain_until  TIMESTAMPTZ,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_documents_entity ON documents.records (tenant_id, entity_type, entity_id);

CREATE TABLE documents.versions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  document_id UUID NOT NULL REFERENCES documents.records(id),
  version     INT NOT NULL,
  object_key  VARCHAR(500) NOT NULL,                -- {tenant}/documents/{id}/v{n}/{file}
  size_bytes  BIGINT NOT NULL,
  sha256      CHAR(64) NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, document_id, version)
);
-- documents.shares (person_id, shared_at, expires_at) and documents.tags
-- (tag_category, tag_value) follow the same tenant_id + document_id pattern.

```

6.12 Domain: Committees (committees.) — R0.2

sql

```

CREATE TABLE committees.records (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  code        VARCHAR(50) NOT NULL,
  name        VARCHAR(200) NOT NULL,
  committee_type VARCHAR(30) NOT NULL,             -- Council, Standing, Temporary, TaskForce
  org_unit_id UUID,                                -- owning organization.units node
  chair_id    UUID,                                -- people.persons
  secretary_id UUID,
  quorum_percentage NUMERIC(5,2) NOT NULL DEFAULT 50.00,
  confidentiality VARCHAR(20) NOT NULL DEFAULT 'Internal',
  is_active     BOOLEAN NOT NULL DEFAULT true,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, code)
);

CREATE TABLE committees.members (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  committee_id UUID NOT NULL REFERENCES committees.records(id),
  person_id   UUID NOT NULL,
  role        VARCHAR(30) NOT NULL,                 -- Chair, Secretary, Member, Observer
  term_start  DATE NOT NULL,
  term_end    DATE,                                -- expiry flips status to 'Expired'
  status      VARCHAR(20) NOT NULL DEFAULT 'Active'
              CHECK (status IN ('Active','Expired','Resigned')),
  has_vote    BOOLEAN NOT NULL DEFAULT true,
  joined_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, committee_id, person_id)
);

```

6.13 Domain: Meetings (meetings.) — R0.2

sql

```
CREATE TABLE meetings.sessions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  committee_id UUID NOT NULL,          -- committees.records
  number      VARCHAR(30),             -- e.g. FC-2026-07
  title       VARCHAR(300) NOT NULL,
  starts_at   TIMESTAMPTZ NOT NULL,
  ends_at     TIMESTAMPTZ,
  location    VARCHAR(200),
  online_url  VARCHAR(500),
  status      VARCHAR(20) NOT NULL DEFAULT 'Scheduled'
  CHECK (status IN ('Scheduled', 'InProgress', 'Concluded', 'Cancelled')),
  quorum_met  BOOLEAN,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_sessions_committee ON meetings.sessions (tenant_id, committee_id, starts_at);

CREATE TABLE meetings.agenda_items (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  meeting_id  UUID NOT NULL REFERENCES meetings.sessions(id),
  "order"     INT NOT NULL,
  title       VARCHAR(300) NOT NULL,
  description  TEXT,                  -- background + proposal
  recommendation TEXT,
  presenter_id UUID,
  document_ids UUID[] NOT NULL DEFAULT '{}', -- documents.records linked to the item
  UNIQUE (tenant_id, meeting_id, "order")
);

CREATE TABLE meetings.attendance (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  meeting_id  UUID NOT NULL REFERENCES meetings.sessions(id),
  person_id   UUID NOT NULL,
  status      VARCHAR(20) NOT NULL CHECK (status IN ('Present', 'Absent', 'Excused')),
  check_in    VARCHAR(10),            -- Manual | QR
  UNIQUE (tenant_id, meeting_id, person_id)
);

CREATE TABLE meetings.votes (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  meeting_id  UUID NOT NULL REFERENCES meetings.sessions(id),
  agenda_item_id UUID NOT NULL REFERENCES meetings.agenda_items(id),
  person_id   UUID NOT NULL,
  choice      VARCHAR(20) NOT NULL CHECK (choice IN ('InFavor', 'Against', 'Abstain')),
  remarks     VARCHAR(500),
  cast_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, meeting_id, agenda_item_id, person_id)
);
-- meetings.minutes_versions: (meeting_id, version, content, status Draft|Review|Approved)
```

6.14 Domain: Decisions (decisions.) — R0.2

sql

```
CREATE TABLE decisions.records (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  number      VARCHAR(30) NOT NULL,    -- e.g. D-2026-041
```

```

    title                VARCHAR(300) NOT NULL,
    decision_type         VARCHAR(30) NOT NULL,          -- Approval, Directive, Recommendation
    authority             VARCHAR(100),                  -- issuing authority / delegated power
    committee_id          UUID,                          -- committees.records
    meeting_id            UUID,                          -- meetings.sessions
    agenda_item_id        UUID,                          -- meetings.agenda_items
    decision_text          TEXT NOT NULL,
    rationale             TEXT,
    owner_id              UUID,                          -- people.persons
    affected_units         UUID[] NOT NULL DEFAULT '{}',
    related_policy_id      UUID,                          -- policies.records
    priority              VARCHAR(20) NOT NULL DEFAULT 'Normal',
    deadline              DATE,
    status                VARCHAR(20) NOT NULL DEFAULT 'Draft'
                        CHECK (status IN ('Draft','Published','Implemented','Verified','Closed')),
    implementation_pct     NUMERIC(5,2) NOT NULL DEFAULT 0,
    decision_date          DATE,
    published_at           TIMESTAMPTZ,
    created_at            TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (tenant_id, number)
);
CREATE INDEX ix_decisions_meeting ON decisions.records (tenant_id, meeting_id);
CREATE INDEX ix_decisions_status ON decisions.records (tenant_id, status);

CREATE TABLE decisions.actions (
    id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id             UUID NOT NULL,
    decision_id           UUID NOT NULL REFERENCES decisions.records(id),
    assignee_id           UUID NOT NULL,                  -- action owner
    org_unit_id           UUID,                          -- affected unit
    description           TEXT NOT NULL,
    status                VARCHAR(20) NOT NULL DEFAULT 'Assigned'
                        CHECK (status IN ('Assigned','InProgress','Done','Verified')),
    due_at               TIMESTAMPTZ NOT NULL,
    task_id               UUID,                          -- tasks.items created on publish
    verifier_id           UUID,
    verified_at           TIMESTAMPTZ
);
CREATE INDEX ix_actions_decision ON decisions.actions (tenant_id, decision_id);

-- implementation_pct = completed (Done/Verified) actions / total actions * 100,
-- recomputed by the Decision-to-Execution Engine on every action status change.

```

6.15 Domain: Policies (policies.) — R0.2

sql

```

CREATE TABLE policies.records (
    id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id             UUID NOT NULL,
    code                  VARCHAR(50) NOT NULL,          -- e.g. POL-IT-003
    title                 VARCHAR(300) NOT NULL,
    content               TEXT NOT NULL,
    status                VARCHAR(20) NOT NULL DEFAULT 'Draft'
                        CHECK (status IN ('Draft','Review','LegalReview','Approval','Published','Retired')),
    version               INT NOT NULL DEFAULT 1,
    owner_id              UUID NOT NULL,                  -- policy owner (people.persons)
    authority             VARCHAR(100),                  -- approving body
    classification         VARCHAR(20) NOT NULL DEFAULT 'Internal',
    effective_date         DATE,
    review_date           DATE,                          -- reminders at 90/60/30 days
    audience              JSONB NOT NULL DEFAULT '{}',   -- {"units":[...],"roles":[...]}
    requires_acknowledgement BOOLEAN NOT NULL DEFAULT false,
    workflow_definition_id UUID,                          -- workflow.definitions binding
    form_id               UUID,                          -- forms.definitions binding
    created_at            TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (tenant_id, code)
)

```

```
);

CREATE TABLE policies.versions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  policy_id   UUID NOT NULL REFERENCES policies.records(id),
  version     INT NOT NULL,
  content     TEXT NOT NULL,
  change_summary TEXT,
  approved_by UUID,
  approved_at TIMESTAMPTZ,
  UNIQUE (tenant_id, policy_id, version)
);

CREATE TABLE policies.acknowledgements (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  policy_id   UUID NOT NULL REFERENCES policies.records(id),
  person_id   UUID NOT NULL,
  policy_version INT NOT NULL,
  status      VARCHAR(20) NOT NULL DEFAULT 'Pending'
    CHECK (status IN ('Pending', 'Acknowledged')),
  at          TIMESTAMPTZ,          -- acknowledgement timestamp
  ip_address  INET,
  UNIQUE (tenant_id, policy_id, person_id)
);
```

6.16 Domain: Notifications (notifications.)

sql

```
CREATE TABLE notifications.queue (
  id          UUID NOT NULL DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  person_id   UUID NOT NULL,
  title       VARCHAR(200) NOT NULL,
  body        TEXT NOT NULL,
  channel     VARCHAR(20) NOT NULL,          -- InApp | Email | Push | Sms | Teams
  priority    VARCHAR(20) NOT NULL DEFAULT 'Normal'
    CHECK (priority IN ('FYI', 'Normal', 'Important', 'Urgent', 'Emergency')),
  status      VARCHAR(20) NOT NULL DEFAULT 'Queued', -- Queued | Sent | Failed
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  read_at     TIMESTAMPTZ,
  PRIMARY KEY (id, status)
) PARTITION BY LIST (status);
CREATE TABLE notifications.queue_queued PARTITION OF notifications.queue FOR VALUES IN ('Queued');
CREATE TABLE notifications.queue_sent PARTITION OF notifications.queue FOR VALUES IN ('Sent');
CREATE TABLE notifications.queue_failed PARTITION OF notifications.queue FOR VALUES IN ('Failed');
CREATE INDEX ix_queue_inbox ON notifications.queue (tenant_id, person_id, status);

CREATE TABLE notifications.templates (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  code        VARCHAR(100) NOT NULL,          -- e.g. task.assigned
  channel     VARCHAR(20) NOT NULL,
  subject     VARCHAR(200) NOT NULL,
  body_template TEXT NOT NULL,              -- ar/en placeholders {{title}}
  is_active   BOOLEAN NOT NULL DEFAULT true,
  UNIQUE (tenant_id, code, channel)
);
```

6.17 Domain: Audit (audit.)

sql

```
CREATE TABLE audit.events (
```

```

id          UUID NOT NULL DEFAULT gen_random_uuid(),
tenant_id   UUID NOT NULL,
action      VARCHAR(100) NOT NULL,          -- what: e.g. approval.approved
entity_type VARCHAR(100) NOT NULL,
entity_id   VARCHAR(100) NOT NULL,          -- e.g. REQ-2026-001245 or UUID
actor_id    UUID,                          -- who (NULL = system)
on_behalf_of UUID,                          -- delegating person, if any
at          TIMESTAMPTZ NOT NULL DEFAULT now(), -- when
ip_address   INET,                          -- where
reason       TEXT,                          -- why
before_state JSONB,
after_state  JSONB,
source       VARCHAR(30) NOT NULL DEFAULT 'api', -- api | workflow | integration
details      JSONB,
PRIMARY KEY (id, at)
) PARTITION BY RANGE (at);
CREATE TABLE audit.events_2026_09 PARTITION OF audit.events
FOR VALUES FROM ('2026-09-01') TO ('2026-10-01');
CREATE INDEX ix_audit_entity ON audit.events (tenant_id, entity_type, entity_id, at);
REVOKE UPDATE, DELETE, TRUNCATE ON audit.events FROM edunexus_app; -- append-only

```

6.18 Domain: Calendar (calendar.) — R0.2

sql

```

CREATE TABLE calendar.events (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  title       VARCHAR(300) NOT NULL,
  event_type  VARCHAR(30) NOT NULL,          -- Meeting, Deadline, PolicyReview, DecisionDeadline,
                                              -- Task, Examination, Training, InstitutionalEvent
  source_type VARCHAR(50),                  -- originating domain, e.g. 'meetings.sessions'
  source_id   UUID,
  starts_at   TIMESTAMPTZ NOT NULL,
  ends_at     TIMESTAMPTZ,
  all_day     BOOLEAN NOT NULL DEFAULT false,
  org_unit_id UUID,                          -- audience scope (NULL = whole tenant)
  location    VARCHAR(200),
  visibility  VARCHAR(20) NOT NULL DEFAULT 'Internal',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, source_type, source_id)
);
CREATE INDEX ix_calendar_range ON calendar.events (tenant_id, starts_at);

```

7. API DESIGN & SPECIFICATIONS

7.1 API Conventions

Convention Standard	
Base URL	/api/v1/
Auth	Bearer JWT in Authorization header
Tenant	Extracted from JWT tenant_id claim (no header needed)
Pagination	?page=1&pageSize=20 → { items, total, page, pageSize }

Convention	Standard
Filtering	?status=Active&priority=Urgent
Sorting	?sortBy=created_at&sortDir=desc
Search	?q=keyword
Language	Accept-Language: ar or en
Error Format	{ type, title, status, detail, errors } (RFC 7807)
Versioning	URL path (/v1/, /v2/)
Rate Limiting	X-RateLimit-Limit, X-RateLimit-Remaining headers

7.2 Complete API Surface

Identity & Access

```

POST    /api/v1/auth/dev-token - Development only: mint HS256 JWT (sub, tenant_id, permission[])
GET     /api/v1/auth/login    - Redirect to Keycloak OIDC authorize endpoint (PKCE)
POST    /api/v1/auth/callback - Exchange OIDC authorization code for access + refresh tokens
POST    /api/v1/auth/refresh  - Rotate refresh token and issue a new access token
POST    /api/v1/auth/logout   - Revoke refresh token and end the Redis-backed session
GET     /api/v1/auth/me       - Current user profile, org unit, roles and effective permissions
POST    /api/v1/auth/mfa/totp/enroll - Enroll a TOTP authenticator for the current user
POST    /api/v1/auth/mfa/verify - Verify a TOTP or SMS OTP challenge
GET     /api/v1/users         - List user accounts (?q=&status=&orgUnitId=)
GET     /api/v1/users/{id}    - User account detail with linked person record
PATCH  /api/v1/users/{id}/status - Activate, suspend or deactivate a user account
GET     /api/v1/roles         - List tenant roles with permission codes (role:read)
POST    /api/v1/roles         - Create role; 409 on duplicate code (role:create)
PUT     /api/v1/roles/{id}/permissions - Replace a role's permission set
POST    /api/v1/roles/assign  - Assign role to person with org scope; SoD conflict → 409
POST    /api/v1/roles/revoke  - Revoke a role assignment (204, RoleRevoked event)
GET     /api/v1/roles/assignments - List role assignments (?personId=)
GET     /api/v1/permissions   - Permission catalog (resource:action codes)
GET     /api/v1/people/{id}/delegations - List authority delegations granted by a person
POST    /api/v1/people/{id}/delegations - Delegate authority to a deputy for a period and scope
DELETE  /api/v1/people/{id}/delegations/{delegationId} - Revoke an active delegation
GET     /api/v1/scim/v2/Users - SCIM 2.0 user listing for HRIS/SIS provisioning
POST    /api/v1/scim/v2/Users - SCIM 2.0 user provisioning
PATCH  /api/v1/scim/v2/Users/{id} - SCIM 2.0 partial update / deprovisioning

```

Organization

```

GET     /api/v1/tenants       - List tenants (platform admin, tenant:read)
POST    /api/v1/tenants       - Create tenant; 409 on duplicate slug (tenant:create)
GET     /api/v1/organizations - List organizations in the tenant (org:read)
POST    /api/v1/organizations - Create organization (university, campus, institute)
GET     /api/v1/organizational-units - List organizational units (faculty, department, center)
POST    /api/v1/organizational-units - Create organizational unit under a parent unit
GET     /api/v1/organizational-units/{id}/subtree - Descendant units via LTREE path query

```

People

```

GET     /api/v1/people        - Directory search (?q=) (person:read)
POST    /api/v1/people        - Create person with unit, position and manager (person:create)

```

```

POST    /api/v1/people/import - Bulk CSV import from HRIS/SIS (?dryRun=true validates only)
GET     /api/v1/people/{id} - Person profile: classification, unit, position, manager, roles
PATCH  /api/v1/people/{id} - Update position, org unit, manager or status
GET     /api/v1/people/{id}/direct-reports - People whose manager is this person

```

Communication

```

GET     /api/v1/communications - List communications (?kind=Announcement|Circular|Directive)
POST    /api/v1/communications - Create draft announcement, circular or directive
GET     /api/v1/communications/{id} - Communication detail with audience and attachments
PATCH  /api/v1/communications/{id} - Edit a draft communication
POST    /api/v1/communications/{id}/audience/preview - Resolve org units + roles to recipients
POST    /api/v1/communications/{id}/publish - Draft → Published; Directive fans out tasks
POST    /api/v1/communications/{id}/read - Record read receipt for the current user
GET     /api/v1/communications/{id}/recipients - Recipient list with read status
GET     /api/v1/communications/{id}/completion - Directive completion dashboard by department
POST    /api/v1/communications/{id}/archive - Archive a published communication
GET     /api/v1/announcements - Published announcements visible to the current user
GET     /api/v1/circulars - Published circulars visible to the current user
POST    /api/v1/chatter/comments - Post comment or @mention on any entity
GET     /api/v1/chatter/comments - Comments for an entity (?entityType=&entityId=)
POST    /api/v1/chatter/follow - Follow an entity to receive its updates
GET     /api/v1/chatter/followers - Followers of an entity (?entityType=&entityId=)

```

Correspondence

```

GET     /api/v1/correspondence - List (?status=); confidential needs extra permission
POST    /api/v1/correspondence - Register correspondence; reserves CORR-yyyy-nnnnnn number
GET     /api/v1/correspondence/{id} - Correspondence detail with attachments and links
POST    /api/v1/correspondence/{id}/submit - Draft → Submitted; creates reviewer approval + task
POST    /api/v1/correspondence/{id}/transition - Advance lifecycle state (Classified, Closed)
GET     /api/v1/correspondence/{id}/thread - Reply/forward thread for a correspondence record
GET     /api/v1/correspondence/{id}/routing-slips - Routing slips (internal referrals) for a record
POST    /api/v1/correspondence/{id}/routing-slips - Refer to a unit or person with instructions
POST    /api/v1/correspondence/{id}/routing-slips/{slipId}/complete - Complete a routing slip
GET     /api/v1/correspondence/{id}/barcode - Barcode/QR image for printed correspondence

```

Inbox

```

GET     /api/v1/inbox - Unified inbox (?filter=all|action|approval|task|request|overdue)
GET     /api/v1/inbox/counts - Unread / action-required counts per item type
POST    /api/v1/inbox/items/{type}/{id}/read - Mark an inbox item as read
GET     /api/v1/inbox/waiting-for - Items the current user submitted that await others
GET     /api/v1/my-work - My Work dashboard: counters, priority work, recent items
POST    /api/v1/hubs/inbox/negotiate - SignalR InboxHub connection (UpdateCount push)

```

Requests

```

GET     /api/v1/requests - List requests/cases (?status=&category=)
POST    /api/v1/requests - Create draft; validates data against form, REQ-yyyy-nnnnnn
GET     /api/v1/requests/{id} - Request detail with form data, workflow state and documents
PATCH  /api/v1/requests/{id} - Edit draft or revise after Changes Requested
POST    /api/v1/requests/{id}/submit - Draft → Submitted; starts workflow or reviewer approval
POST    /api/v1/requests/{id}/attachments - Chunked upload for attachments > 10 MB (virus scanned)
POST    /api/v1/requests/{id}/cancel - Withdraw a request before final decision
GET     /api/v1/requests/categories - Request categories with default form and workflow

```

Forms

```

GET     /api/v1/forms - List form definitions (form:read)
POST    /api/v1/forms - Create form definition; unique code per tenant (form:manage)
GET     /api/v1/forms/{id} - Form definition with current JSON schema
PUT     /api/v1/forms/{id} - Update draft schema: sections, fields, rules, conditions
POST    /api/v1/forms/{id}/publish - Publish schema as a new immutable version
GET     /api/v1/forms/{id}/versions - Version history of a form schema
POST    /api/v1/forms/{id}/validate - Dry-run validation of submission data against schema
GET     /api/v1/forms/field-types - Supported field types and their validation options
POST    /api/v1/form-submissions - Submit form data bound to a schema version

```

GET /api/v1/form-submissions/{id} – Submission with the schema version it was captured on

Workflows

GET /api/v1/workflows/definitions – List workflow definitions
POST /api/v1/workflows/definitions – Create definition; validates node graph (workflow:manage)
GET /api/v1/workflows/definitions/{id} – Definition with nodes, transitions and SLA config
PUT /api/v1/workflows/definitions/{id} – Update draft definition from the visual designer
POST /api/v1/workflows/definitions/{id}/validate – Validate graph: End reachable, no orphans
POST /api/v1/workflows/definitions/{id}/publish – Publish a new definition version
GET /api/v1/workflows/instances – List instances (?status=Running|Completed|Rejected)
POST /api/v1/workflows/instances – Start an instance manually for an entity
GET /api/v1/workflows/instances/{id} – Instance state with current node and context
GET /api/v1/workflows/instances/{id}/history – Node-by-node execution history
POST /api/v1/workflows/instances/{id}/cancel – Cancel a running instance (admin)

Approvals

GET /api/v1/approvals – List approvals (?assigneeId=&status=) (approval:read)
GET /api/v1/approvals/{id} – Approval detail with entity, documents and history
POST /api/v1/approvals/{id}/decide – Approve or reject; assignee only, 409 on double decide
POST /api/v1/approvals/{id}/request-changes – Return to requester (ChangesRequested)
POST /api/v1/approvals/{id}/delegate – Reassign approval and linked task to a deputy
GET /api/v1/approvals/{id}/history – Decision trail incl. delegated authority
GET /api/v1/approvals/submitted – Approvals the current user is waiting on

Tasks

GET /api/v1/tasks – List tasks (?assigneeId=&status=) (task:read)
POST /api/v1/tasks – Create a manual task
GET /api/v1/tasks/{id} – Task detail with dependencies, evidence and comments
PATCH /api/v1/tasks/{id} – Update description, priority, progress (0–100) or status
POST /api/v1/tasks/{id}/complete – Mark task completed (task:update)
POST /api/v1/tasks/{id}/verify – Verifier accepts or rejects completion evidence
GET /api/v1/tasks/{id}/comments – Task discussion
POST /api/v1/tasks/{id}/comments – Add a comment to a task
GET /api/v1/tasks/{id}/evidence – Evidence files attached to a task
POST /api/v1/tasks/{id}/evidence – Attach evidence (object key + file name)
POST /api/v1/activities – Schedule a follow-up activity on any entity
GET /api/v1/activities/my – Current person's scheduled activities (?completed=)
PATCH /api/v1/activities/{id}/complete – Mark a scheduled activity done

SLA & Escalation

GET /api/v1/sla/breaches – Breached approvals and tasks (approval:read)
GET /api/v1/sla/policies – SLA policies (durations, reminder and escalation levels)
POST /api/v1/sla/policies – Create an SLA policy for a workflow, category or task type
GET /api/v1/sla/compliance – SLA compliance rate by unit and period
GET /api/v1/escalations – Escalation log (reminder, manager, executive levels)
POST /api/v1/escalations/{id}/acknowledge – Escalation owner acknowledges and takes action

Documents

GET|POST /api/v1/document-workspaces – List or create document workspaces
GET|POST /api/v1/documents – List or create document records (document:read/create)
GET|PATCH /api/v1/documents/{id} – Document detail / update name, classification, status
POST /api/v1/documents/{id}/upload-url – Presigned MinIO/S3 upload URL for a new version
POST /api/v1/documents/{id}/versions – Register an uploaded version (DocumentVersionAdded)
POST /api/v1/documents/{id}/share – Share with person/unit (view, comment, edit)
GET /api/v1/documents/{id}/shares – List active shares of a document
GET|POST /api/v1/documents/{id}/tags – Read or add document tags (may fire action rules)
GET|POST /api/v1/documents/{id}/retention-policy – Read or set the retention policy
GET|POST /api/v1/documents/action-rules – List or create tag-driven document action rules
DELETE /api/v1/documents/action-rules/{ruleId} – Delete a document action rule

Committees (R0.2)

GET /api/v1/committees – List committees visible to the user (committee:read)

```

POST    /api/v1/committees - Create committee with type, quorum and confidentiality
GET     /api/v1/committees/{id} - Committee detail
PATCH  /api/v1/committees/{id} - Update mandate, quorum percentage or status
GET     /api/v1/committees/{id}/workspace - Aggregated meetings, members, decisions and documents
GET     /api/v1/committees/{id}/members - Members with role and term dates
POST    /api/v1/committees/{id}/members - Add member with role (chair, secretary, member)
DELETE  /api/v1/committees/{id}/members/{memberId} - End a membership
POST    /api/v1/committees/{id}/terms - Open a new committee term

```

Meetings (R0.2)

```

GET     /api/v1/meetings - List meetings (?committeeId=) (meeting:read)
POST    /api/v1/meetings - Schedule a meeting for a committee (MeetingScheduled)
GET     /api/v1/meetings/{id} - Meeting detail
PATCH  /api/v1/meetings/{id} - Reschedule or change location / online link
GET     /api/v1/meetings/{id}/agenda - Agenda with ordered items and linked documents
POST    /api/v1/meetings/{id}/agenda-items - Submit an agenda item
PATCH  /api/v1/agenda-items/{id} - Edit or reorder an agenda item
POST    /api/v1/agenda-items/{id}/documents - Link a document to an agenda item
POST    /api/v1/meetings/{id}/agenda/submit - Secretary submits agenda for chair approval
POST    /api/v1/meetings/{id}/agenda/approve - Chair approves agenda; members are notified
GET     /api/v1/meetings/{id}/packet - Meeting packet: agenda, papers, prior actions
POST    /api/v1/meetings/{id}/attendance - Record attendance (manual check-in)
POST    /api/v1/meetings/{id}/check-in - QR-code self check-in
GET     /api/v1/meetings/{id}/quorum - Quorum status: present / total vs. quorum %
GET     /api/v1/meetings/{id}/votes - Votes cast in the meeting
POST    /api/v1/meetings/{id}/votes - Cast a vote on an agenda item
POST    /api/v1/meetings/{id}/decisions - Record a decision from the meeting (DecisionPublished)
GET     /api/v1/meetings/{id}/minutes - Minutes with version history
POST    /api/v1/meetings/{id}/minutes - Save a new minutes draft version
POST    /api/v1/minutes/{id}/approve - Approve minutes version (draft → review → approved)
POST    /api/v1/meetings/{id}/conclude - Conclude the meeting (MeetingConcluded)

```

Decisions (R0.2)

```

GET     /api/v1/decisions - List decisions (?meetingId=&status=) (decision:read)
GET     /api/v1/decisions/{id} - Decision record with actions, evidence and implementation %
POST    /api/v1/decisions/{id}/transition - Next lifecycle state (Approved, Published, Closed)
POST    /api/v1/decisions/{id}/actions - Assign an action to an owner (ActionAssigned)
GET     /api/v1/decisions/{id}/implementation - Implementation % = completed / total actions
POST    /api/v1/decision-actions/{id}/advance - Advance action status (ActionAdvanced)
POST    /api/v1/decision-actions/{id}/evidence - Attach implementation evidence to an action
POST    /api/v1/decision-actions/{id}/verify - Designated verifier accepts or rejects evidence
GET     /api/v1/decisions/overdue - Decisions with overdue actions
GET     /api/v1/decisions/{id}/memory - Linked meeting, agenda item, policy and documents

```

Policies (R0.2)

```

GET     /api/v1/policies - List policies (?status=) (policy:read)
POST    /api/v1/policies - Create a draft policy (policy:create)
GET     /api/v1/policies/{id} - Policy detail with current version and bindings
POST    /api/v1/policies/{id}/versions - Create a new revision draft
POST    /api/v1/policies/{id}/transition - Draft → Review → LegalReview → Approval
POST    /api/v1/policies/{id}/publish - Publish; resolves audience, creates acknowledgements
POST    /api/v1/policies/{id}/acknowledge - Acknowledge; timestamp + IP recorded (policy:ack)
GET     /api/v1/policies/{id}/acknowledgements - Acknowledgement completion percentage
GET     /api/v1/policies/{id}/pending - Active staff who have not acknowledged
POST    /api/v1/policies/{id}/retire - Retire a published policy
GET     /api/v1/policies/reviews/upcoming - Policies due for review in 30/60/90 days
GET     /api/v1/procedures - Procedures linked to policies

```

Calendar (R0.2)

```

GET     /api/v1/calendar/events - Institutional calendar (?from=&to=&type=)
POST    /api/v1/calendar/events - Create an institutional event
PATCH  /api/v1/calendar/events/{id} - Update an institutional event
DELETE  /api/v1/calendar/events/{id} - Remove an institutional event
GET     /api/v1/calendar/export.ics - iCalendar feed of the user's calendar

```

Notifications

GET	/api/v1/notifications	– Notifications for a person (?personId=) (notification:read)
POST	/api/v1/notifications/{id}/read	– Mark a notification as read
GET	/api/v1/notifications/preferences	– Channel, quiet hours, digest and language preferences
PUT	/api/v1/notifications/preferences	– Update notification preferences
GET	/api/v1/notification-templates	– List templates (notification:manage)
POST	/api/v1/notification-templates	– Create template; unique code + channel, {token} rendering
GET	/api/v1/notification-receipts	– Per-channel delivery receipts log

Reports & Dashboards

GET	/api/v1/analytics/overview	– Executive KPIs: requests, approvals, overdue, SLA breaches
GET	/api/v1/dashboards/executive	– Executive dashboard tiles
GET	/api/v1/dashboards/department/{orgUnitId}	– Department completion and workload
GET	/api/v1/dashboards/governance	– Governance dashboard (R0.2)
GET	/api/v1/reports/my-performance	– Personal throughput and on-time rate
GET	/api/v1/reports/workflow-cycle-time	– Cycle time by workflow and step
GET	/api/v1/reports/sla-compliance	– SLA compliance by unit and period
GET	/api/v1/reports/approval-bottlenecks	– Slowest approval steps and approvers
GET	/api/v1/reports/committees	– Committee activity, attendance, quorum compliance (R0.2)
GET	/api/v1/reports/decisions	– Decisions by period, authority, implementation (R0.2)
GET	/api/v1/reports/policies	– Policy inventory, reviews, acknowledgement compliance (R0.2)
GET	/api/v1/reports/{code}/export	– Export a report (?format=xlsx pdf csv)

Audit

GET	/api/v1/audit	– Audit trail (?entityType=&from=&to=&limit=, max 1000) (audit:read)
GET	/api/v1/audit/{id}	– Audit event with before/after snapshot
GET	/api/v1/audit/entities/{type}/{id}	– Audit history of one business record
GET	/api/v1/audit/export	– Export audit events for a period (CSV)

Search / Institutional Memory

GET	/api/v1/search	– Cross-domain search (?q=, min 2 chars) (search:read)
GET	/api/v1/search/suggest	– Type-ahead suggestions (?q=)
GET	/api/v1/search/memory/{entityType}/{id}	– Institutional memory: linked records graph

AI (R0.2)

POST	/api/v1/ai/index	– Build per-tenant OpenSearch index, Arabic analyzer (ai:manage)
POST	/api/v1/ai/ask	– Governance query answered from authorized data with sources
GET	/api/v1/ai/interactions	– AI interaction log for review (ai:read)
POST	/api/v1/ai/meetings/{id}/summary	– Summarize pending actions from previous 3 meetings
POST	/api/v1/ai/meetings/{id}/draft-minutes	– Draft minutes from notes; pending secretary review
POST	/api/v1/ai/minutes/{id}/extract-decisions	– Extract decisions and actions; pending review

8. MODULE TECHNICAL SPECIFICATIONS

8.1 M01 — Identity & Organization

Key Technical Details:

- Multi-tenant via tenant_id column + Row-Level Security (RLS) in PostgreSQL
- Organization hierarchy uses PostgreSQL LTREE extension for efficient ancestor/descendant queries
- User provisioning: SCIM 2.0 endpoint for HRIS/SIS integration
- Session management: Redis-backed with sliding expiration (30 min default)

- MFA: TOTP (Google Authenticator) + SMS OTP

RLS Policy Example:

sql

```
ALTER TABLE identity.users ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON identity.users
USING (tenant_id = current_setting('app.current_tenant', true)::uuid);
```

8.2 M02 — Communication Center

Key Technical Details:

- Rich text editor (TipTap) with Arabic RTL support
- Target audience resolver: evaluates org hierarchy + roles to determine recipients
- Directive → Task auto-creation: when requires_action = true, the system publishes a DirectivePublished event → Task Service creates tasks for all target users
- Notification fan-out: one announcement → N notification queue entries (batched)

8.3 M03 — Correspondence

Key Technical Details:

- Auto-numbering: configurable per tenant (e.g., CORR-2026-{sequence})
- Barcode/QR generation for printed correspondence
- OCR integration (future): extract metadata from scanned documents
- State machine for correspondence lifecycle (Received → Registered → Classified → Assigned → Action → FollowUp → Response → Closed)
- Full-text search on subject + body via OpenSearch

8.4 M04 — Unified Inbox

Key Technical Details:

- Aggregation query across multiple tables (correspondence, approvals, tasks, requests, notifications)
- Materialized view or read model in PostgreSQL, refreshed via CDC (Change Data Capture) events
- Real-time updates via SignalR hub: InboxHub.UpdateCount(userId)
- Filter implementation: single endpoint with type discriminator

8.5 M05 — Request Center

Key Technical Details:

- Dynamic form rendering: frontend receives JSON schema → renders form components
- Category-based routing: each category maps to a default workflow
- Case number auto-generation with tenant-specific patterns
- File upload: chunked upload for large files (>10MB), virus scan hook

8.6 M06 — Dynamic Form Builder

Key Technical Details:

- Form schema stored as JSONB in PostgreSQL
- Supported field types mapped to React components:

Field Type	Backend Validation	Frontend Component
Text	MaxLength, Regex	TextInput
Long text	MaxLength	TextArea
Number	Min, Max, Decimal	NumberInput
Currency	Min, Max, DecimalPlaces	CurrencyInput
Date	MinDate, MaxDate	DatePicker
Date/time	MinDateTime	DateTimePicker
Dropdown	AllowedValues	Select
Multi-select	AllowedValues	MultiSelect
Radio	AllowedValues	RadioGroup
Checkbox	Required	Checkbox
User	ValidUserId	UserPicker
Department	ValidOrgId	OrgPicker
Organization	ValidOrgId	OrgTreePicker
Document	FileTypes, MaxSize	DocumentUpload
Attachment	FileTypes, MaxSize	FileUpload

Field Type	Backend Validation	Frontend Component
Signature	Required	SignaturePad
Table/grid	RowValidation	DataGrid

- Conditional logic engine: evaluates if/then rules on the client side, validated on server
- Form versioning: each publish creates a new version; existing submissions retain their schema version

8.7 M07 — Workflow Engine (see Section 9)

8.8 M08 — Approval Center

Key Technical Details:

- Approval record captures: decision, timestamp, user, org, comments, delegation info, documents, audit event
- Parallel approval: waits for all approvers (configurable: all-must-approve vs. majority)
- Delegation check: before presenting approval, check identity.delegations for active delegation
- Timeout: if approval not acted on within configured hours → escalation event

8.9 M09 — Tasks & Actions

Key Technical Details:

- Task sources: Workflow, Decision, Directive, Manual
- Progress tracking: manual (user updates %) or automatic (based on subtasks)
- Dependency graph: topological sort for task ordering
- Evidence collection: file upload + description + timestamp
- Verification workflow: task owner marks complete → verifier reviews → verified/rejected

8.10 M10 — SLA & Escalation (see Section 9.4)

8.11 M11 — Documents (see Section 11)

8.12 M12 — Audit (see Section 14.3)

8.13 M13 — Committees (R0.2)

Key Technical Details:

- Committee workspace: aggregated view of meetings, members, decisions, documents
- Term management: automatic member status update when term expires
- Quorum calculation: $\text{present_members} / \text{total_members} \geq \text{quorum_percentage}$
- Confidentiality enforcement: ABAC policy checks committee confidentiality vs. user clearance

8.14 M14 — Meetings (R0.2)

Key Technical Details:

- Agenda builder: drag-and-drop ordering, linked documents per item
- Digital meeting room: real-time collaboration via SignalR (chat, voting, decision recording)
- Attendance tracking: manual check-in or QR code scan
- Minutes versioning: draft → review → approved (each version stored)
- AI-assisted minutes generation (see Section 13)

8.15 M15 — Decisions (R0.2)

Key Technical Details:

- Decision-to-Execution Engine: when decision status changes to Published, the system:
 1. Reads affected_units and actions from the decision record
 2. Creates decisions.actions records
 3. Creates tasks.items for each action
 4. Sets SLA deadlines
 5. Sends notifications to all assigned owners
- Implementation tracking: $\text{implementation_percentage} = (\text{completed_actions} / \text{total_actions}) * 100$
- Verification: designated verifier reviews evidence for each action

8.16 M16 — Policies (R0.2)

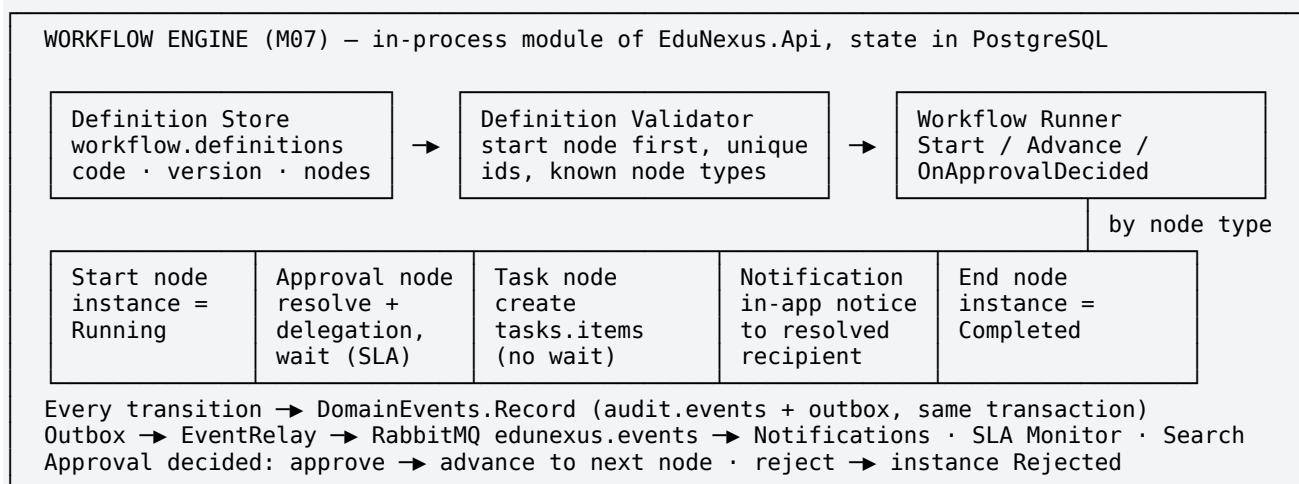
Key Technical Details:

- Policy lifecycle state machine: Draft → Review → LegalReview → Approval → Published → Retired

- Policy-to-Workflow binding: a policy can reference a workflow definition, form, and approval matrix
- Acknowledgement engine:
 1. On publish, resolve audience (org units + roles)
 2. Create policies.acknowledgements records for each target user (status: Pending)
 3. Send notification with link to policy
 4. User clicks "Acknowledge" → timestamp + IP recorded
 5. Dashboard shows completion percentage
- Review calendar: automated reminder 30/60/90 days before review_date

9. WORKFLOW ENGINE — TECHNICAL DESIGN

9.1 Architecture



9.2 Node Executor Pattern

csharp

```

public interface INodeExecutor
{
    string NodeType { get; } // start | approval | task | notification | end
    Task<NodeResult> ExecuteAsync(NodeContext ctx, JsonElement node, CancellationToken ct);
}
public sealed record NodeResult(NodeOutcome Outcome, Guid? PendingApprovalId = null);
public enum NodeOutcome { Continue, WaitForApproval, Completed }
public sealed class ApprovalNodeExecutor(IAssigneeResolver resolver) : INodeExecutor
{
    public string NodeType => "approval";
    public async Task<NodeResult> ExecuteAsync(
        NodeContext ctx, JsonElement node, CancellationToken ct)
    {

```

```

        // personId | assigneeFrom (submitter, department_head, dean) | roleCode, then delegation
        var assignee = await resolver.ResolveAsync(ctx, node, ct);
        var approval = ctx.CreateApproval(assignee, slaDays: node.GetInt32OrDefault("slaDays", 5));
        ctx.RecordEvent("WorkflowAdvanced", node.GetProperty("id").GetString()!, approval.Id);
        return new NodeResult(NodeOutcome.WaitForApproval, approval.Id);
    }
}

```

9.3 Workflow Definition JSON Schema

json

```

{
  "code": "HR-LEAVE",
  "name": "Staff Leave Request",
  "version": 3,
  "isActive": true,
  "trigger": { "entityType": "Request", "category": "HR" },
  "nodes": [
    { "id": "start", "type": "start" },
    {
      "id": "head_review",
      "type": "approval",
      "assigneeFrom": "department_head",
      "slaDays": 2
    },
    {
      "id": "dean_approval",
      "type": "approval",
      "assigneeFrom": "dean",
      "slaDays": 3
    },
    {
      "id": "hr_processing",
      "type": "task",
      "roleCode": "hr.officer",
      "title": "Record approved leave in HR system"
    },
    { "id": "notify_submitter", "type": "notification", "assigneeFrom": "submitter",
      "text": "Your leave request has been approved." },
    { "id": "end", "type": "end" }
  ]
}

```

9.4 SLA & Escalation Engine

Implementation:

- Hangfire recurring job checks every 5 minutes for SLA breaches
- Redis sorted set for fast SLA timer lookups: ZADD sla:timers {deadline_timestamp} {task_id}
- Escalation chain stored as JSONB in workflow definition:

json

```

{
  "slaPolicy": "TASK-DEFAULT",
  "escalation": [
    { "level": 1, "afterHours": 0, "action": "mark_breached", "notify": ["assignee"] },
    { "level": 2, "afterHours": 48, "action": "remind", "notify": ["assignee"] },
    { "level": 3, "afterHours": 72, "action": "reassign", "to": "escalation_owner",
      "notify": ["escalation_owner", "assignee"] },
  ]
}

```

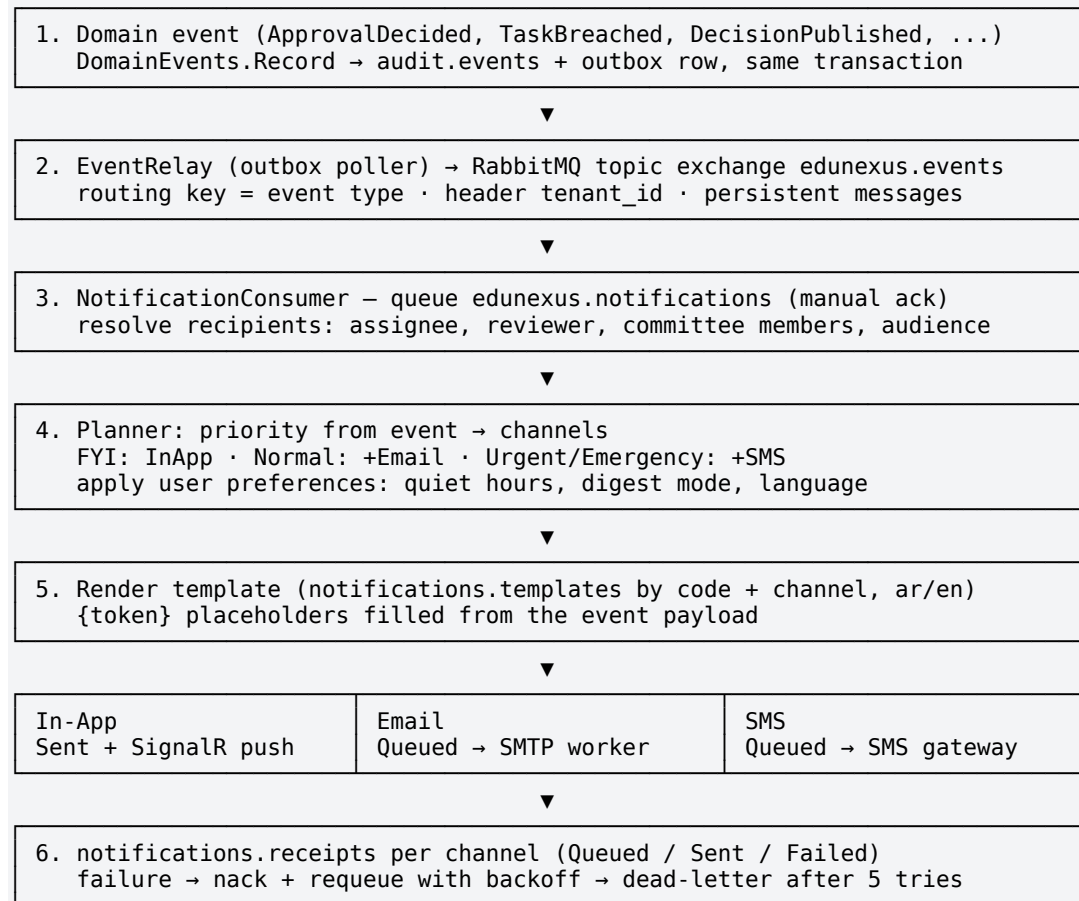
```

    { "level": 4, "afterHours": 120, "action": "notify", "to": "submitter_dean",
      "notify": ["submitter_dean"] }
  ]
}

```

10. NOTIFICATION ENGINE — TECHNICAL DESIGN

10.1 Pipeline



10.2 Real-Time Delivery

csharp

```

[Authorize]
public sealed class NotificationHub : Hub
{
    public override async Task OnConnectedAsync()
    {
        var group = $"user:{Context.User!.TenantId()}:{Context.UserId}";
        await Groups.AddToGroupAsync(Context.ConnectionId, group);
        await base.OnConnectedAsync();
    }
}

// NotificationConsumer, after the InApp row commits (status = Sent):
await hub.Clients.Group($"user:{n.TenantId}:{n.PersonId}")
    .SendAsync("notification", new { n.Id, n.Title, n.Body, n.Priority, n.CreatedAt }, ct);
await hub.Clients.Group($"user:{n.TenantId}:{n.PersonId}").SendAsync("inboxCount", unread, ct);

```

10.3 User Preferences

Each user can configure:

- Channel preferences per notification type
 - Quiet hours (no push/SMS between 10 PM – 7 AM)
 - Digest mode (batch notifications into daily email)
 - Language preference (Arabic / English)
-

11. DOCUMENT MANAGEMENT — TECHNICAL DESIGN

11.1 Upload Pipeline

```
1. POST /api/documents → documents.records (status Draft, current_version 0)
2. POST /api/documents/{id}/upload-url → presigned PUT URL (expires in 1 hour)
   object key = {tenant_id}/{document_id}/{uuid}-{file_name}
```



```
3. Browser PUTs the file directly to S3 / MinIO (API never proxies file bytes)
```



```
4. POST /api/documents/{id}/versions {objectKey, sizeBytes, sha256, publish}
   → documents.versions row · current_version + 1 · DocumentVersionAdded event
5. Async consumers: virus scan → text extraction (OCR) → OpenSearch → preview
```

11.2 Storage Structure (S3)

```
edunexus-docs/                                one bucket; tenant isolation by key prefix
├── {tenant_id}/
│   ├── {document_id}/
│   │   ├── 3f2a9c41d7e84b10a6c2e5f9b0d1e7a2-Self_Study_Report.pdf    version 1
│   │   └── 9b71d0e6a3c94f2b8e57d1a04c6b2f93-Self_Study_Report.pdf    version 2 (current)
│   ├── evidence/{task_id}/{uuid}-{file_name}                        task evidence
│   ├── attachments/{entity_type}/{entity_id}/{uuid}-{file_name}     request/correspondence
│   ├── previews/{document_id}/v{version}/page-{n}.png               generated, CDN cached
│   └── exports/{yyyy}/{mm}/{report_id}.xlsx                         report exports
```

Object keys are immutable: a new upload always creates a new key + documents.versions row.
Bucket policy: private; access only via presigned URLs (PUT/GET, 1 hour expiry).

11.3 Access Control

- Document access follows transaction-level permissions
 - Classification-based restrictions: Confidential documents require explicit role
 - Share links: time-limited, password-protected, view-only
 - Watermarking: configurable for sensitive documents
-

12. SEARCH & INSTITUTIONAL MEMORY

12.1 OpenSearch Index Strategy

Index	Source Tables	Key Fields
correspondence	correspondence.records	subject, body, source, number
requests	requests.cases	title, category, number
tasks	tasks.items	title, description
decisions	decisions.records	title, decision_text, number
policies	policies.records	title, content
documents	documents.records	name, extracted_text
meetings	meetings.sessions + agendas	title, agenda items
announcements	communication.announcements	title, body

12.2 Arabic Search Support

json

```
{
  "settings": {
    "analysis": {
      "filter": {
        "arabic_stop": { "type": "stop", "stopwords": "_arabic_" },
        "arabic_stemmer": { "type": "stemmer", "language": "arabic" } },
      "analyzer": {
        "content_analyzer": {
          "type": "custom", "tokenizer": "standard",
          "filter": ["lowercase", "arabic_normalization", "arabic_stop", "arabic_stemmer"]
        }
      }
    }
  }
}
```

12.3 Institutional Memory Query

Query: "accreditation" (tenant-scoped, trimmed to the user's permissions and clearance)

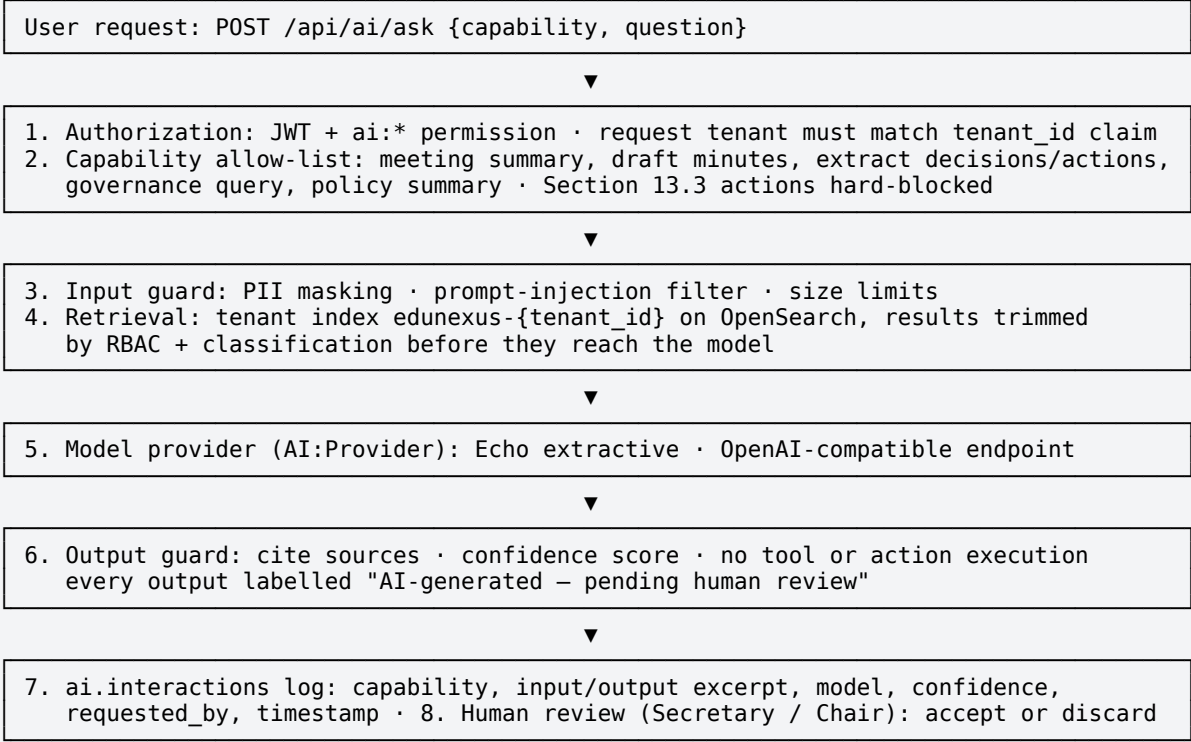
- Correspondence CORR-2026-0142 "NCAAA accreditation site visit schedule"
 - Request REQ-2026-0310 "Prepare self-study evidence"
 - Task "Compile program KPIs" → Evidence: KPI_Report_2026.pdf
- Meeting "Faculty Council – Session 7" (Committee: Faculty Council)
 - Decision DEC-2026-0021 "Approve accreditation self-study plan"
 - Action → Task "Update course specifications" Done · Verified
 - Action → Task "Collect student survey results" In progress · 60%
- Policy POL-QA-004 "Program Quality Assurance Policy" v3 · Published
- Announcement "Accreditation readiness workshop"
- Documents 12 matches (self-study drafts, evidence files)

Timeline view: Correspondence → Request → Meeting → Decision → Actions → Tasks → Evidence

Implementation: graph traversal via source_id / transaction_id foreign keys + OpenSearch cross-index query.

13. AI INTEGRATION ARCHITECTURE

13.1 Guardrail Architecture



13.2 AI Capabilities (R0.2)

Capability	Input	Output	Human Review
Meeting Summary	Previous 3 meetings' minutes	Summary text	Yes (Secretary)
Draft Minutes	Meeting transcript/notes	Structured minutes	Yes (Secretary)
Extract Decisions	Minutes text	List of decisions	Yes (Chair)
Extract Actions	Decision text	List of actions + owners	Yes (Secretary)
Governance Query	Natural language question	Data-backed answer with sources	N/A (read-only)

Capability	Input	Output	Human Review
Policy Summary	Policy document	Plain-language summary	N/A (read-only)

13.3 AI Restrictions (Hardcoded)

The AI service **must not**:

- Approve institutional decisions
- Change grades or academic records
- Issue disciplinary decisions
- Publish binding policies
- Alter financial authority
- Override workflow permissions
- Modify user roles or permissions

All AI outputs are marked as AI-generated — pending human review.

14. SECURITY ARCHITECTURE

14.1 Defense in Depth

Layer	Controls
Network	VPC, private subnets, WAF, DDoS protection
Transport	TLS 1.3 everywhere, HSTS
Application	Input validation, parameterized queries, CSRF tokens, XSS sanitization
Authentication	OIDC/OAuth2, MFA, session timeout, brute-force protection
Authorization	RBAC + ABAC + Org Scope + Classification
Data	Encryption at rest (AES-256), encryption in transit, PII masking
Audit	Immutable audit log, separate from application logs
Infrastructure	Container scanning, secret management (Vault/AWS Secrets), least-privilege IAM

14.2 Confidentiality Levels

Level	Access Rule	Example
Public	Anyone in tenant	General announcements
Internal	All authenticated users	Internal circulars
Confidential	Specific roles + org scope	HR cases, procurement
Restricted	Named individuals + committee members	Disciplinary, ethics
Highly Confidential	Executive + legal only	Legal proceedings

14.3 Audit Implementation

What gets audited (business events only, NOT UI clicks):

- All CRUD on governance objects (committees, meetings, decisions, policies)
- All approval actions (approve, reject, delegate)
- All workflow state transitions
- All correspondence lifecycle changes
- All task completions and verifications
- All document classifications and access changes
- All policy acknowledgements
- All delegation creations and revocations
- All login/logout events
- All permission changes

Audit event format:

json

```
{
  "id": "0b8e6c1a-5d2f-4c7e-9a31-7f4e2d9b6c10",
  "tenantId": "7c1e4a92-3b5d-4f08-9e26-1a4d8c7b3f55",
  "action": "ApprovalDecided",
  "entityType": "Approval",
  "entityId": "e4f1a7c3-92b6-4d18-a0c5-6b3e8f2d9a47",
  "actorId": "5a9d2e71-0c84-4b3f-b6e2-9f1c7a4d8e03",
  "at": "2026-03-14T09:42:17.381Z",
  "details": "approved",
  "ipAddress": "10.20.4.17",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736",
```

```
"changes": { "status": { "from": "Pending", "to": "Approved" } }
```

Storage: Append-only table with no UPDATE/DELETE permissions for application role. Separate read-only replica for compliance reporting.

15. PERFORMANCE, SCALABILITY & CACHING

15.1 Caching Strategy

Data	Cache Layer	TTL	Invalidation
User profile + roles	Redis	15 min	On role change
Org hierarchy tree	Redis	1 hour	On org change
Form definitions	Redis	30 min	On form publish
Workflow definitions	Redis	30 min	On workflow activate
Inbox counts	Redis	1 min	On new item
Dashboard KPIs	Redis	5 min	Scheduled refresh
Notification rules	Redis	10 min	On rule change
Policy content	Redis	1 hour	On version publish
Document previews	CDN / S3	24 hours	On new version

15.2 Database Optimization

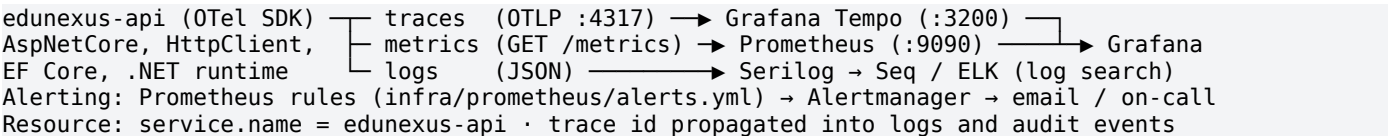
- **Indexes:** B-tree for lookups, GIN for JSONB, GIST for LTREE, BRIN for timestamps
- **Partitioning:** audit.events partitioned by month; notifications.queue partitioned by status
- **Connection pooling:** PgBouncer in front of PostgreSQL (transaction mode)
- **Read replicas:** 1 read replica for reporting queries
- **Query optimization:** EF Core compiled queries for hot paths

15.3 Performance Targets

Metric	Target
API response time (p95)	< 200ms
Dashboard load time	< 2s
Search query time	< 500ms
File upload (10MB)	< 5s
Workflow state transition	< 1s
Notification delivery (in-app)	< 2s
Notification delivery (email)	< 30s
Concurrent users	1,000+ (initial)

16. MONITORING & OBSERVABILITY

16.1 OpenTelemetry Stack



16.2 Key Metrics to Monitor

Category	Metrics
API	Request rate, latency (p50/p95/p99), error rate, 4xx/5xx breakdown
Database	Connection pool usage, query latency, slow queries, deadlocks
Queue	Message rate, queue depth, consumer lag, dead-letter count
Cache	Hit rate, memory usage, eviction rate
Search	Index size, query latency, indexing rate
Storage	Upload rate, storage growth, failed uploads
Business	Active workflow instances, pending approvals, overdue tasks, SLA breaches, decision implementation rate

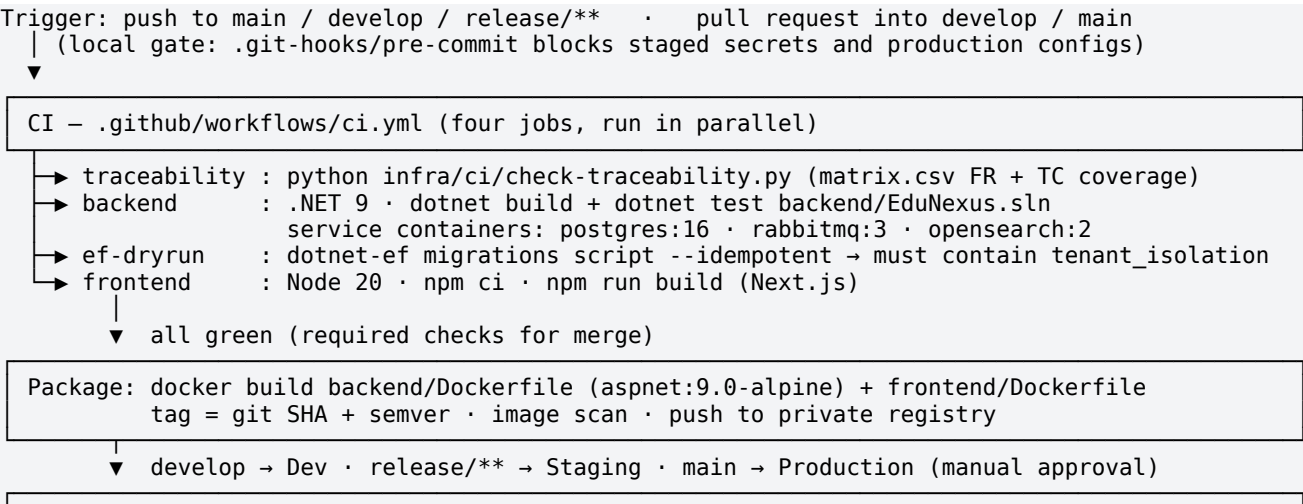
Category	Metrics
Infrastructure	CPU, memory, disk, network per pod/node

16.3 Alerting Rules

Alert	Condition	Severity
High error rate	5xx > 1% for 5 min	Critical
API latency spike	p99 > 2s for 5 min	Warning
Database connection exhaustion	Pool > 90%	Critical
Queue backlog	Depth > 10,000 for 10 min	Warning
SLA breach surge	> 50 breaches/hour	Warning
Disk space	> 85% used	Warning
Certificate expiry	< 14 days	Warning

17. CI/CD & DEVOPS PIPELINE

17.1 Pipeline Stages



```
Deploy: helm upgrade --install edunexus infra/helm/edunexus
pre-upgrade hook Job edunexus-migrate: dotnet EduNexus.Api.dll --migrate-only
rolling update of API pods · readiness probe on /health
```



```
Verify: smoke tests (/health, login, inbox) · Playwright E2E (nightly + pre-release)
k6 load (infra/k6) against SLA targets · Grafana error/latency watch 30 min
```

17.2 Database Migration Strategy

- EF Core migrations versioned in source control
- Migrations run automatically on deploy (idempotent)
- Breaking changes require multi-step migration (add column → backfill → remove old)
- Rollback plan: maintain backward-compatible schema for at least 2 releases

18. TESTING STRATEGY

18.1 Test Pyramid

Level	Tools	Coverage Target	Execution
Unit	xUnit, Moq, FluentAssertions, Jest, React Testing Library	80%+	Every PR
Integration	Testcontainers (PostgreSQL, Redis, RabbitMQ), WebApplicationFactory	Key flows	Every PR
E2E	Playwright	Critical user journeys	Nightly + pre-release
Performance	k6	SLA targets	Pre-release
Security	OWASP ZAP, Snyk	OWASP Top 10	Weekly

18.2 Critical Test Scenarios

R0.1:

1. User creates request → form renders dynamically → submits → workflow starts → approval routed → task created → SLA timer starts → notification sent → evidence uploaded → task verified → audit trail complete
2. Dean creates directive circular → tasks auto-created for all department heads → deadlines enforced → escalations fire → completion dashboard updates

3. Correspondence received → registered → classified → assigned → responded → closed → searchable

R0.2:

1. Secretary creates committee meeting → agenda built → chair approves → members notified → meeting conducted → attendance recorded → minutes drafted → decisions extracted → actions assigned → departments execute → evidence collected → implementation verified → governance dashboard updated
 2. Policy drafted → reviewed → approved → published → acknowledgement sent to 1,240 users → 94.5% acknowledge → compliance report generated
 3. Institutional memory search: "accreditation" returns related communications, meetings, decisions, policies, tasks, and documents
-

19. DATA MIGRATION & MULTI-TENANCY

19.1 Multi-Tenancy Model

Shared Database, Schema-per-Tenant (via RLS)

```
JWT tenant_id → TenantScope.BeginAsync → set_config('app.current_tenant', tenant_id, true)
PostgreSQL 16 · one database · schema per domain
Every tenant-owned table: tenant_id uuid NOT NULL + ENABLE / FORCE ROW LEVEL SECURITY
POLICY tenant_isolation USING / WITH CHECK
    (tenant_id = current_setting('app.current_tenant', true)::uuid)
platform tables (tenants, outbox) carry no RLS policy
```

19.2 Tenant Onboarding Process

1. Create tenant → POST /api/tenants {slug, name} → tenants row (platform level, no RLS)
2. Identity → OIDC realm/client (Keycloak or customer IdP) with tenant_id claim mapper
3. Security → seed roles + permission catalog, assign tenant administrator
4. Organization → import organizations, campuses, units (CSV/Excel, parent-child check)
5. People → import people + role assignments (CSV or LDAP/AD sync)
6. Configuration → numbering patterns, form library, workflow definitions, templates
7. Platform → OpenSearch index edunexus-{tenant_id}, object prefix {tenant_id}/ in S3
8. Verification → login, org tree, sample request → approval → task → notification
9. Go-live → sign-off, historical data import (Section 19.3), hypercare 2 weeks

19.3 Data Import Tools

- **Org hierarchy:** CSV/Excel import with parent-child validation
 - **Users:** CSV import + LDAP/AD sync
 - **Correspondence:** CSV import for historical records
 - **Documents:** Bulk upload with metadata CSV
-

20. RELEASE PHASING & DEPENDENCIES

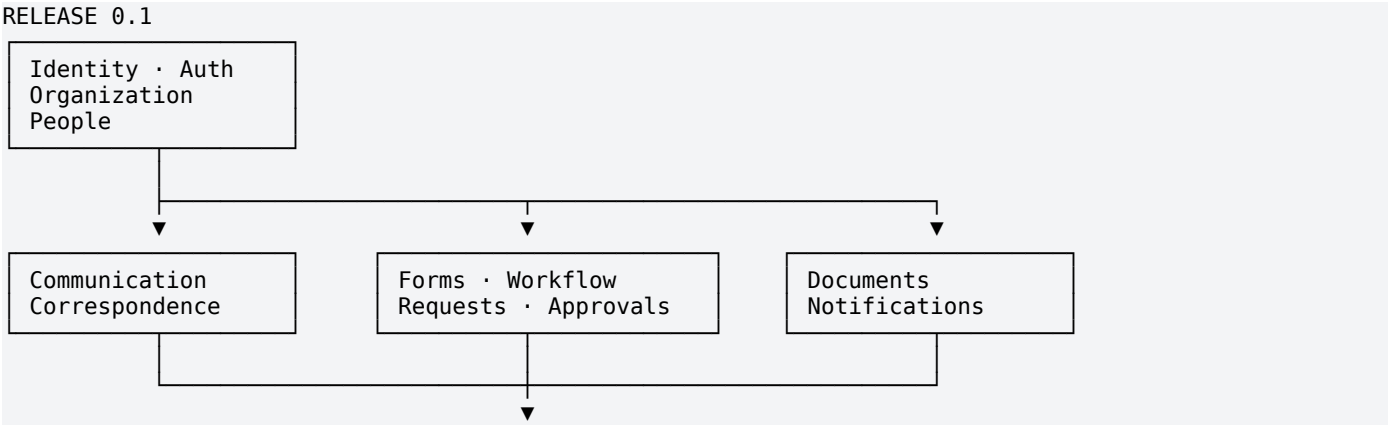
20.1 Release 0.1 — Phased Delivery (16 weeks)

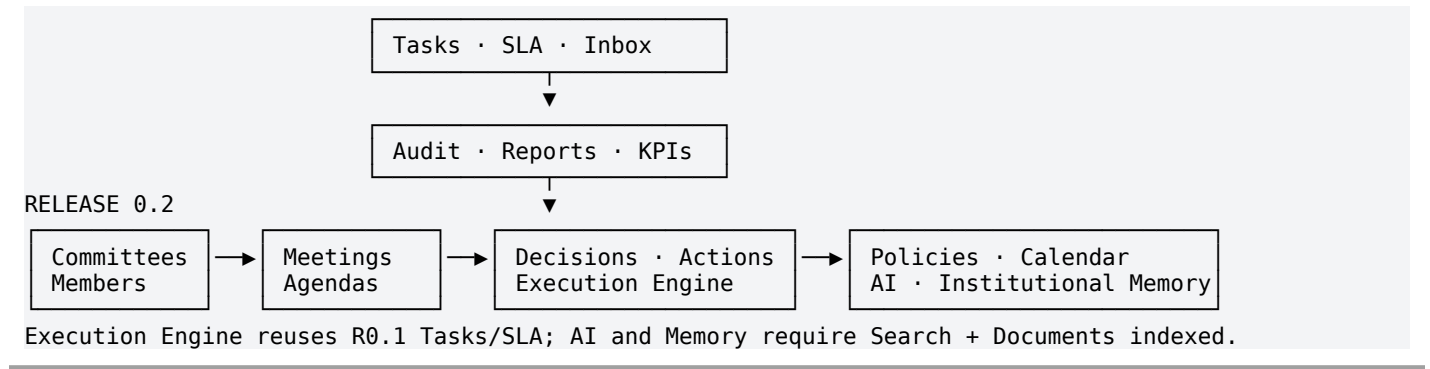
Phase	Weeks	Modules	Milestone
Phase 1: Foundation	1–4	Identity, Organization, People, Auth	Users can log in and see org tree
Phase 2: Communication	5–7	Announcements, Circulars, Correspondence, Inbox	Users can send/receive communications
Phase 3: Transactions	8–11	Requests, Forms, Workflow, Approvals	End-to-end request → approval flow
Phase 4: Execution	12–14	Tasks, SLA, Documents, Notifications	Tasks assigned, SLA enforced, evidence collected
Phase 5: Reporting	15–16	Audit, Reports, Dashboards	Executive dashboards live

20.2 Release 0.2 — Phased Delivery (12 weeks)

Phase	Weeks	Modules	Milestone
Phase 1: Committees	1–3	Committees, Members, Workspaces	Committee management operational
Phase 2: Meetings	4–6	Meetings, Agendas, Attendance, Minutes	Full meeting lifecycle
Phase 3: Decisions	7–9	Decisions, Actions, Execution Engine	Decision-to-execution pipeline
Phase 4: Governance	10–12	Policies, Calendar, AI, Institutional Memory	Governance dashboard + AI assistant

20.3 Dependency Graph





21. NON-FUNCTIONAL REQUIREMENTS

Category	Requirement	Target
Availability	Uptime SLA	99.9% (excluding maintenance windows)
Performance	API p95 latency	< 200ms
Performance	Dashboard load	< 2s
Scalability	Concurrent users	1,000+ (R0.1), 5,000+ (R0.2)
Scalability	Data volume	10M+ audit events/year
Security	Authentication	OIDC/OAuth2 + MFA
Security	Data encryption	AES-256 at rest, TLS 1.3 in transit
Security	Penetration testing	Annual + pre-release
Compliance	Data residency	Configurable per tenant
Compliance	Audit retention	7 years minimum
i18n	Languages	Arabic (RTL) + English (LTR)
i18n	Date/number formats	Hijri + Gregorian
Accessibility	WCAG	2.1 AA
Backup	RPO	< 1 hour
Backup	RTO	< 4 hours
Disaster Recovery	Failover	Multi-AZ (production)
Browser Support	Minimum	Chrome 90+, Edge 90+, Firefox 90+,

Category	Requirement	Target
		Safari 14+
Mobile	PWA	Installable, offline-capable for read

22. DEFINITION OF DONE

22.1 Release 0.1 Definition of Done

The release is **complete** when an institution can perform this entire journey inside EduNexus:

Create communication → target users → require action → generate work → route approval → assign task → enforce SLA → send notifications → collect evidence → verify completion → produce audit trail → report results.

Specific acceptance criteria:

- 12 core modules deployed and functional
- 2 killer workflows demonstrated end-to-end (Faculty Administrative Request + Executive Circular → Action)
- Arabic/English UI fully functional with RTL support
- PWA installable and functional on mobile
- All API endpoints documented (OpenAPI/Swagger)
- Unit test coverage > 80%
- Integration tests pass for all critical flows
- E2E tests pass for both killer workflows
- Security scan clean (no critical/high vulnerabilities)
- Performance benchmarks met
- Audit trail captures all business events
- Executive, department, and employee dashboards functional
- Multi-tenant isolation verified
- Disaster recovery tested
- User documentation complete
- Admin deployment guide complete

22.2 Release 0.2 Definition of Done

The release is **complete** when an institution can execute this scenario entirely inside EduNexus:

Create committee → schedule meeting → build agenda → distribute papers → conduct meeting → record attendance → create minutes → approve minutes → issue decisions → automatically assign actions → track execution → collect evidence → verify implementation → report institutional performance.

Specific acceptance criteria:

- 8 governance modules deployed and functional
 - Faculty Council Decision workflow demonstrated end-to-end
 - Decision-to-Execution engine auto-creates tasks from decisions
 - Policy lifecycle with acknowledgement tracking functional
 - AI Meeting Assistant generates draft minutes (human-reviewed)
 - AI Governance Query answers questions from institutional data
 - Institutional Memory search returns cross-entity results
 - Governance dashboard shows decision implementation rates
 - Institutional calendar consolidates all event types
 - Security upgrade: RBAC + ABAC + classification enforcement
 - All R0.1 acceptance criteria still passing (regression)
-

APPENDIX A: Project Structure

```
lms/
├── .github/workflows/ci.yml          # CI: traceability, backend, ef-dryrun, frontend
├── .git-hooks/pre-commit            # blocks staged secrets / production configs
├── OWNERS.md
├── README.md
├── backend/
│   ├── Dockerfile                  # sdk:9.0-alpine build → aspnet:9.0-alpine runtime
│   ├── EduNexus.sln
│   └── src/
│       ├── EduNexus.Api/           # ASP.NET Core 9 minimal-API host
│       │   ├── Program.cs          # DI, auth, OpenTelemetry, endpoint mapping
│       │   ├── Program.Partial.cs
│       │   ├── appsettings.json
│       │   ├── appsettings.Development.json
│       │   ├── EduNexus.Api.http    # request samples
│       │   ├── AI/
│       │   │   ├── TenantSearchIndex.cs # per-tenant OpenSearch index + AiOptions
│       │   └── Auth/
│       │       └── AuthOptions.cs      # OIDC authority / audience
```

- └─ DevTokenService.cs # dev-only HS256 tokens
- └─ Endpoints/
 - └─ ApiDtos.cs
 - └─ EndpointHelpers.cs
 - └─ FoundationEndpoints.cs # tenants, orgs, units, people, roles
 - └─ CorrespondenceEndpoints.cs
 - └─ GovernanceEndpoints.cs # committees, meetings, decisions, policies
 - └─ DocumentEndpoints.cs # documents, versions, upload URLs
 - └─ IntelligenceEndpoints.cs # AI, approvals, tasks, notifications
- └─ Events/
 - └─ EventRelay.cs # outbox → RabbitMQ edunexus.events
 - └─ NotificationConsumer.cs
 - └─ NotificationPlanner.cs # priority → channel matrix
 - └─ RabbitMqOptions.cs
 - └─ SlaMonitor.cs # breach → reminder → escalation
- └─ Integrations/
 - └─ IntegrationDispatcher.cs # outbound webhooks
 - └─ WebhookSigner.cs
- └─ Properties/launchSettings.json
- └─ Storage/ObjectStorage.cs # MinIO / S3 presigned URLs
- └─ EduNexus.Foundation/ # domain records, enums, validators
 - └─ Foundation.cs
 - └─ FoundationStore.cs
 - └─ FormValidation.cs
 - └─ WorkflowDefinitionValidator.cs
- └─ EduNexus.Infrastructure/ # EF Core persistence
 - └─ AppDbContext.cs
 - └─ AppDbContextFactory.cs
 - └─ DomainEvents.cs # audit + outbox in one transaction
 - └─ Sequences.cs # tenant numbering
 - └─ TenantScope.cs # app.current_tenant for RLS
 - └─ WorkflowRunner.cs
 - └─ Migrations/ # InitialFoundation, RlsPolicies, Outbox, R2Communication ... R14EnterpriseAutomation
- └─ tests/
 - └─ EduNexus.Api.Tests/ # WebApplicationFactory integration tests
 - └─ EduNexusFactory.cs
 - └─ FoundationApiTests.cs
 - └─ EduNexus.Foundation.Tests/
- └─ frontend/ # Next.js App Router + TypeScript
 - └─ Dockerfile
 - └─ package.json
 - └─ next.config.ts
 - └─ playwright.config.ts
 - └─ app/
 - └─ layout.tsx
 - └─ page.tsx # dashboard
 - └─ approvals/page.tsx
 - └─ audit/page.tsx
 - └─ correspondence/page.tsx
 - └─ directory/page.tsx
 - └─ intelligence/page.tsx
 - └─ meetings/page.tsx
 - └─ roles/page.tsx
 - └─ components/ # AppLayout, Chatter, ConnectionBar, TranslationProvider
 - └─ lib/ # api.ts, translations.ts (ar/en)
 - └─ scripts/capture-guide-screenshots.js
 - └─ tests/e2e/home.spec.ts
- └─ infra/
 - └─ docker-compose.yml # postgres, redis, rabbitmq, minio, opensearch, prometheus, grafana, tempo, keycloak (idp profile)
 - └─ ci/check-traceability.py
 - └─ grafana/ # datasources, dashboards, dashboard-api.json
 - └─ helm/edunexus/ # Chart.yaml, values.yaml, AIRGAP.md
 - └─ templates/ # api.yaml, stateful.yaml, migrate-job.yaml
 - └─ k6/ # r1-load.js, write-mix-r2-r5.js
 - └─ k8s/namespace.yaml
 - └─ keycloak/ # setup.ps1, roles-proof.ps1, README.md

```

├── postgres/                # RUNBOOK.md, restore-snippet.conf
├── prometheus/              # prometheus.yml, alerts.yml
├── scripts/sis_erp_sync.py
├── tempo/tempo.yaml
├── docs/
│   ├── 01-master-blueprint/
│   ├── 02-frs/
│   ├── 03-architecture/    # adr/, security-baseline.md, secrets-management.md
│   ├── 04-ux/
│   ├── 05-processes/
│   ├── 06-data/
│   ├── 07-api/
│   ├── 08-analytics/
│   ├── 09-ai/
│   ├── 10-roadmap/          # README.md, GA-CHECKLIST.md
│   ├── 11-traceability/    # matrix.csv (BR → FR → API → TC)
│   ├── assets/
│   └── User_Guide.md
├── tests/k6/                # load.js, run-k6.ps1

```

APPENDIX B: Key Integration Points (Future)

System	Integration Method	Purpose
SIS (Student Information System)	REST API / Webhook	Student enrollment, grades, schedules
LMS (Learning Management System)	LTI / REST API	Course creation from decisions
ERP / Finance	REST API / SOAP	Procurement, budget approvals
HRIS	SCIM 2.0 / REST API	Employee directory sync
Active Directory / LDAP	LDAP / SAML	Authentication federation
Microsoft 365 / Teams	Microsoft Graph API	Calendar sync, Teams notifications
Email (Exchange / Gmail)	IMAP / SMTP / Graph	Correspondence integration
Accreditation Systems	REST API	Quality data export

This document serves as the comprehensive technical blueprint for building EduNexus OS Releases 0.1 and 0.2. It should be used alongside the product backlog (250+ user stories) and screen-by-screen specifications to guide engineering execution from blueprint → product → production.

1/2

both

Thinking completed

Understood. Based on the strategic direction outlined in the document, the logical next step is to deliver **both** the **Unified Product Backlog** (Epics, User Stories, Acceptance Criteria) and the **Screen-by-Screen Product Specification** for Releases 0.1 and 0.2.

This bridges the gap between the high-level technical architecture and actual engineering execution.

PART 1: UNIFIED PRODUCT BACKLOG (Releases 0.1 & 0.2)

Priority Key: P0 = Critical for MVP/Killer Workflow, P1 = High Value, P2 = Nice to Have

Module Group A: Foundation & Identity (M01)

ID	Epic	User Story	Acceptance Criteria	Priority
A1	Tenant & Org Setup	As an Admin, I want to define the organizational hierarchy (University → Faculty → Department) so that routing and permissions work correctly.	Hierarchy supports infinite nesting; LTREE path generated; UI displays as a collapsible tree.	P0
A2	User Provisioning	As an Admin, I want to import users via CSV or SCIM so that they have accounts, roles, and managers assigned.	Bulk import validates data; users receive welcome email; manager field links to directory.	P0
A3	Delegation Rules	As a Manager, I want to delegate my approval authority to a deputy for a specific date range so that work doesn't bottleneck during my absence.	Delegation is time-bound; delegated actions show "Approved by [Delegate] on behalf of [Manager]" in audit log.	P1

Module Group B: Communication & Inbox (M02, M04)

ID	Epic	User Story	Acceptance Criteria	Priority
B1	Circulars vs. Directives	As a Dean, I want to issue a Directive that automatically generates tasks for target audiences, unlike a standard	"Requires Action" toggle creates tasks in the background; deadline and	P0

ID	Epic	User Story	Acceptance Criteria	Priority
		Announcement.	escalation rules are inherited.	
B2	Unified Inbox	As an Employee, I want a single inbox filtering Correspondence, Approvals, Tasks, and Requests so I know exactly what requires my attention.	Filter tabs work instantly; badge counts update in real-time via SignalR; "Action Required" is the default view.	P0

Module Group C: Requests & Dynamic Forms (M05, M06)

ID	Epic	User Story	Acceptance Criteria	Priority
C1	Dynamic Form Builder	As an Admin, I want to build a form with conditional logic (e.g., if "IT" is selected, show "Hardware/Software" dropdown) without coding.	Form schema saved as JSONB; frontend renders components dynamically; server validates schema on submission.	P0
C2	Request Submission	As an Employee, I want to submit a "Faculty Administrative Request" with attachments and see a case number immediately.	Case number auto-generated (e.g., REQ-2026-001); file upload supports chunking; success screen shows next steps.	P0

Module Group D: Workflow, Approvals & Tasks (M07, M08, M09, M10)

ID	Epic	User Story	Acceptance Criteria	Priority
D1	Visual Workflow Designer	As an Admin, I want to drag-and-drop nodes (Start, Form, Approval, Task, End) to define a business process.	React Flow integration; JSON export validates against backend schema; versioning supported.	P0
D2	Approval Actions	As an Approver, I want to Approve, Reject, Request Changes, or Delegate with mandatory comments for rejection.	Audit log captures decision, timestamp, user, IP, and comments; workflow state transitions immediately.	P0
D3	SLA & Escalation	As a System, I want to escalate a task to a manager if it remains "In Progress" past 72 hours.	Background worker checks SLA timers; escalation triggers notification and updates task	P1

ID	Epic	User Story	Acceptance Criteria	Priority
			owner; visible in audit trail.	
D4	Task Evidence	As a Task Owner, I want to upload evidence (documents, screenshots) and mark a task as "Completed" for verification.	Evidence linked to task record; status changes to "Waiting Verification"; verifier receives notification.	P0

Module Group E: Governance - Committees & Meetings (M13, M14) Release 0.2

ID	Epic	User Story	Acceptance Criteria	Priority
E1	Committee Workspace	As a Committee Secretary, I want a dedicated workspace showing members, upcoming meetings, and pending actions.	Dashboard shows quorum %, implementation rate, and next meeting date.	P0
E2	Agenda Builder	As a Secretary, I want to build an agenda with background docs, proposals, and recommendations for each item.	Drag-and-drop ordering; each item can be flagged as "For Decision".	P0
E3	Digital Meeting Room	As a Committee Member, I want to view the agenda, mark my attendance, and record my vote on decisions during the meeting.	Real-time attendance tracking; voting options (Approve/Reject/Defer); chat sidebar.	P1

Module Group F: Governance - Decisions & Policies (M15, M16) Release 0.2

ID	Epic	User Story	Acceptance Criteria	Priority
F1	Decision-to-Execution Engine	As a Dean, I want an approved decision to automatically generate actionable tasks for affected departments with deadlines.	Decision approval triggers background job; tasks created and linked to Decision ID; implementation % calculated dynamically.	P0

ID	Epic	User Story	Acceptance Criteria	Priority
F2	Policy Acknowledgement	As an Employee, I want to be notified of a new policy, read it, and click "Acknowledge" so the institution can track compliance.	Acknowledgement records user ID, timestamp, IP, and policy version; dashboard shows % compliance.	P1

Module Group G: Institutional Memory & AI (M17, M18) Release 0.2

ID	Epic	User Story	Acceptance Criteria	Priority
G1	Cross-Entity Search	As a User, I want to search "postgraduate program" and see related communications, meetings, decisions, and policies.	OpenSearch queries across all indexed domains; results grouped by entity type with source attribution.	P1
G2	AI Meeting Assistant	As a Secretary, I want the AI to draft meeting minutes and extract decisions/actions from my rough notes for my review.	AI output is clearly marked as "Draft"; human must explicitly click "Approve & Publish" before it becomes official.	P2

PART 2: SCREEN-BY-SCREEN PRODUCT SPECIFICATION

This defines the UI/UX flow for the **Two Killer Workflows**, ensuring the product delivers on its core value proposition.

Screen 1: Login & Authentication

- **Components:** Logo, Language Toggle (EN/AR), Username/Email input, Password input, "Forgot Password" link, MFA prompt (if enabled).
- **State:** On success, redirects to /my-work. JWT stored in secure HTTP-only cookie.
- **Business Rule:** Failed attempts > 5 trigger a 15-minute account lockout.

Screen 2: Home / "My Work" Dashboard (The Core Experience)

- **Header:** Greeting ("Good morning, Dr. Ahmed"), Faculty/Dept context, Global Search, Notification Bell (with unread count), Profile Menu.
- **Top KPI Cards:**
 - Tasks (e.g., 12)

- o Approvals (e.g., 4)
 - o Requests (e.g., 7)
 - o Overdue (e.g., 2, highlighted in red)
- **Priority Work List:**
 - o Row 1: "Approve faculty workload" | Due: Today | [Action Button]
 - o Row 2: "Reply to correspondence #CORR-882" | Due: Today | [Action Button]
- **Recent Communications:** Feed of latest Announcements/Circulars targeted to the user's org unit.

Screen 3: Unified Inbox

- **Layout:** Left sidebar (Filters: All, Action Required, Approval, Correspondence, Task, Request, Overdue). Main panel lists items.
- **Item Row:** Icon (type), Title, Sender, Date, Priority Badge, Status Badge, Quick Action Button (e.g., "Review").
- **Interaction:** Clicking an item opens a side-panel or modal with full details and action buttons without leaving the inbox context.

Screen 4: New Request (Dynamic Form)

- **Step 1:** Category Selection (Grid of icons: Academic, HR, Finance, IT, etc.).
- **Step 2:** Dynamic Form Render.
 - o *Example (IT Request):* Requester (auto-filled), Purpose (Text), Required Date (DatePicker), Items (Data Grid: Item, Qty, Cost), Attachments (Drag & Drop).
- **Step 3:** Preview & Submit. Shows the mapped workflow (e.g., "This will be routed to: Dept Head → Finance → Dean").
- **Post-Submit:** Success toast with Case Number (e.g., REQ-2026-001245) and link to track progress.

Screen 5: Approval Center

- **Context Panel:** Shows request details, submitted form data, and attached documents (with inline PDF preview).
- **Workflow Timeline:** Visual stepper showing current stage (e.g., "Pending: Dean Approval").
- **Action Bar:**

- [Approve] → Opens optional comment modal → Submits.
- [Reject] → **Mandatory** comment field → Submits (triggers rejection path in workflow).
- [Request Changes] → Routes back to requester with comments.
- [Delegate] → Opens user picker to select delegate for this specific action.

Screen 6: Task Execution & Evidence

- **Task Details:** Title, Description, Owner, Due Date, SLA countdown timer.
- **Progress Section:** Slider or dropdown to update status (New → In Progress → Waiting → Completed).
- **Evidence Section:** "Upload Evidence" button. Lists previously uploaded files.
- **Action:** [Mark as Completed] → Changes status to "Waiting Verification" → Notifies the verifier.

Screen 7: Committee Workspace & Meeting Room (*Release 0.2*)

- **Workspace Header:** Committee Name, Chair, Next Meeting Date, KPIs (Pending Actions, Implementation %).
- **Tabs:** Overview | Meetings | Members | Decisions | Documents.
- **Meeting Room View (Active Meeting):**
 - Left: Agenda Tree (expandable items with Background, Proposal, Recommendation).
 - Center: Document Preview (PDF viewer).
 - Right: Attendance Checklist (Present/Absent/Late) + Live Chat/Discussion notes.
 - Bottom: Decision Recording Panel → [Propose Decision] → [Vote: Approve/Reject/Defer].

Screen 8: Decision Execution Dashboard (*Release 0.2*)

- **Header:** Decision Number (e.g., D-2026-041), Title, Date, Status (e.g., "In Execution").
- **Implementation Tracker:**
 - Progress Bar: 72% Complete
 - Action List:
 -  Curriculum Committee: Review curriculum (Completed)

-  Quality Unit: Accreditation alignment (Completed)
 -  Registrar: Create course in SIS (Due in 5 days)
 -  IT: Create LMS course (Overdue by 2 days) → [Escalate] button visible to Dean.
- **Evidence Tab:** Aggregates all documents uploaded by task owners against this specific decision.
-

PART 3: DEFINITION OF DONE (DoD) & ENGINEERING SEQUENCING

To move from blueprint to production, the engineering team must adhere to this sequencing and DoD.

Development Sequencing (Sprints 1-12)

1. **Sprint 1-2:** Foundation (M01 Identity, Org, People) + Auth + Basic UI Shell.
2. **Sprint 3-4:** Communication (M02) + Unified Inbox (M04) + Notifications (M10).
3. **Sprint 5-6:** Requests (M05) + Dynamic Forms (M06) + Document Upload (M11).
4. **Sprint 7-8:** Workflow Engine (M07) + Approvals (M08) + Audit (M12).
5. **Sprint 9-10:** Tasks (M09) + SLA/Escalation (M10) + Dashboards/Reports. *(R0.1 Feature Complete)*
6. **Sprint 11-12:** Committees (M13) + Meetings (M14) + Decisions & Execution Engine (M15). *(R0.2 Core Complete)*

Definition of Done (Per User Story)

- Code reviewed and merged to main.
- Unit tests written (>80% coverage for business logic).
- Integration tests pass (API + DB).
- UI matches Figma/design spec (including RTL Arabic support).
- Audit event logged for state-changing actions.
- API documented in Swagger/OpenAPI.
- Acceptance criteria verified by Product Owner.

Definition of Done (Release 0.1)

The system successfully executes the **Faculty Administrative Request** killer workflow end-to-end in the staging environment, with all SLA timers, notifications, and audit trails functioning, and zero P0/P1 bugs.

Definition of Done (Release 0.2)

The system successfully executes the **Faculty Council Decision → Automatic Action Assignment → Evidence Verification** killer workflow, with the Governance Dashboard accurately reflecting the implementation percentage.

Next Immediate Action for the Team:

1. **Product:** Finalize Figma wireframes for Screens 2, 4, 5, and 8 (the core value screens).
2. **Engineering:** Initialize the monorepo, set up the Docker Compose local environment (Postgres, Redis, RabbitMQ, MinIO), and implement the M01 Identity/Org database schemas.
3. **QA:** Draft the end-to-end Playwright test scripts for the two killer workflows to ensure test-driven development.

Would you like me to expand on any specific module (e.g., the exact JSON schema for the Dynamic Form Builder, or the database migration script for the Decision-to-Execution engine)?